ĐẠI HỌC QUỐC GIA TPHCM

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN

KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO ĐỒ ÁN

Chủ đề: Lab 01: Hadoop Cluster & WordCount

Môn học: Nhập môn Dữ liệu lớn

Sinh viên thực hiện:

23120180 - Trần Lê Trung Trực

Giảng viên hướng dẫn:

Thầy Trần Huy Bân

Ngày 1 tháng 4 năm 2026

Mục lục

1 Cài đặt Hadoop Pseudo-Distributed **1**

1.1 Môi trường hệ thống và địa chỉ IP . . . 1

1.2 Cài đặt Java và Scala . . . 2

1.3 Cài đặt SSH . . . 2

1.4 Tải và giải nén Hadoop . . . 3

1.5 Cấu hình biến môi trường . . . 4

1.6 Cấu hình XML (core-site.xml, hdfs-site.xml) . . . 6

1.7 Format NameNode . . . 7

1.8 Khởi động Cluster và kiểm tra tiến trình . . . 8

1.9 Kiểm tra WebUI . . . 8

1.10 Tạo thư mục HDFS và User Linux . . . 9

1.11 Tải dữ liệu lên HDFS . . . 10

1.12 Phân quyền và chuyển Owner . . . 10

1.13 Chạy file JAR xác thực . . . 11

2 WordCount bằng Scala **13**

2.1 Giới thiệu bài toán . . . 13

2.2 Bước 1: Cài đặt SBT và cấu hình build.sbt . . . 13

2.3 Bước 2: Cấu hình YARN và MapReduce . . . 15

2.4 Bước 3: Chuẩn bị dữ liệu đầu vào . . . 15

2.5 Bước 4: Mã nguồn Scala (WordCount.scala) . . . 16

2.6 Bước 5: Biên dịch và xử lý sự cố . . . 19

2.7 Bước 6: Trích xuất và kiểm tra kết quả . . . 24

3 Cài đặt Cụm Hadoop Fully-Distributed **24**

3.1 Giai đoạn 1: Chuẩn bị môi trường Docker . . . 24

3.2 Giai đoạn 2: Cấu hình SSH không mật khẩu . . . 25

3.3 Giai đoạn 3: Cài đặt Hadoop trên Master . . . 26

3.4 Giai đoạn 4: Cấu hình các file XML cốt lõi . . . 27

3.5 Giai đoạn 5: Nhân bản cấu hình sang Worker 28
3.6 Giai đoạn 6: Xử lý sự cố phân quyền Root 29
3.7 Giai đoạn 7: Khởi chạy cụm và kiểm tra 30

4 Kết luận **31**

Danh sách hình vẽ

1 Địa chỉ IP của máy trong WSL Ubuntu 22.04 1
2 Cài đặt Java 11 và Scala 2.12.18 thành công 2
3 Cấu hình SSH thành công - kết nối localhost không cần mật khẩu 3
4 Tải và kiểm tra bộ cài Hadoop 3.3.6 4
5 Cấu hình biến môi trường trong /.bashrc và hadoop-env.sh 5
6 Lệnh hadoop version trả về Hadoop 3.3.6 5
7 Nội dung cấu hình core-site.xml và hdfs-site.xml 7
8 Format NameNode thành công - thông báo "successfully formatted" 7
9 Khởi động DFS, YARN và kiểm tra tiến trình JPS đủ 5 daemon 8
10 WebUI Hadoop tại localhost:9870 - tab Datanodes hiển thị 1 node In service 9
11 Tạo thư mục HDFS /hcmus/23120180 và user Linux khtn_23120180 9
12 Tải file words.txt lên HDFS và xác nhận bằng lệnh ls 10
13 Phân quyền chmod 744 và chuyển owner cho thư mục và file bên trong 11
14 Chạy hadoop-test.jar thành công và tạo file xác thực 12
15 Nội dung file 23120180_verification.txt 12
16 Cài đặt SBT thành công, lệnh `sbt -version` trả về phiên bản đã cài 14
17 Cấu hình file `build.sbt` khai báo thư viện Hadoop 14
18 Nội dung cấu hình `yarn-site.xml` và `mapred-site.xml` 15
19 Biên dịch Fat JAR bằng `sbt assembly` thành công 20
20 Quá trình tra cứu và giải quyết lỗi NoClassDefFoundError trên diễn đàn StackOverflow 21
21 Lỗi `MRAppMaster`: YARN ApplicationMaster không tìm thấy classpath của MapReduce 22
22 Quá trình tra cứu và giải quyết lỗi MRAppMaster trên diễn đàn StackOverflow . . 23
23 Job MapReduce hoàn thành thành công: `map 100% reduce 100%` 23
24 Nội dung file `results.txt` - định dạng TSV, sắp xếp đúng thứ tự f, i, t, h, c, m, u, s 24
25 Khởi tạo 3 container Docker thành công 25
26 Kiểm tra đăng nhập SSH không mật khẩu từ Master đến Worker1 và Worker2 thành công 26
27 Lệnh `hadoop version` xác nhận đã cài Hadoop 3.3.6 thành công trên container Master 27
28 Lỗi phân quyền Root: Hadoop 3.x mặc định cấm chạy dịch vụ bằng quyền root . . . 29

29 Quá trình tra cứu và giải quyết lỗi phân quyền Root trên diễn đàn StackOverflow . 30
30 Kết quả lệnh `jps` trên cả 3 node: Master, Worker1 và Worker2 31

1 Cài đặt Hadoop Pseudo-Distributed

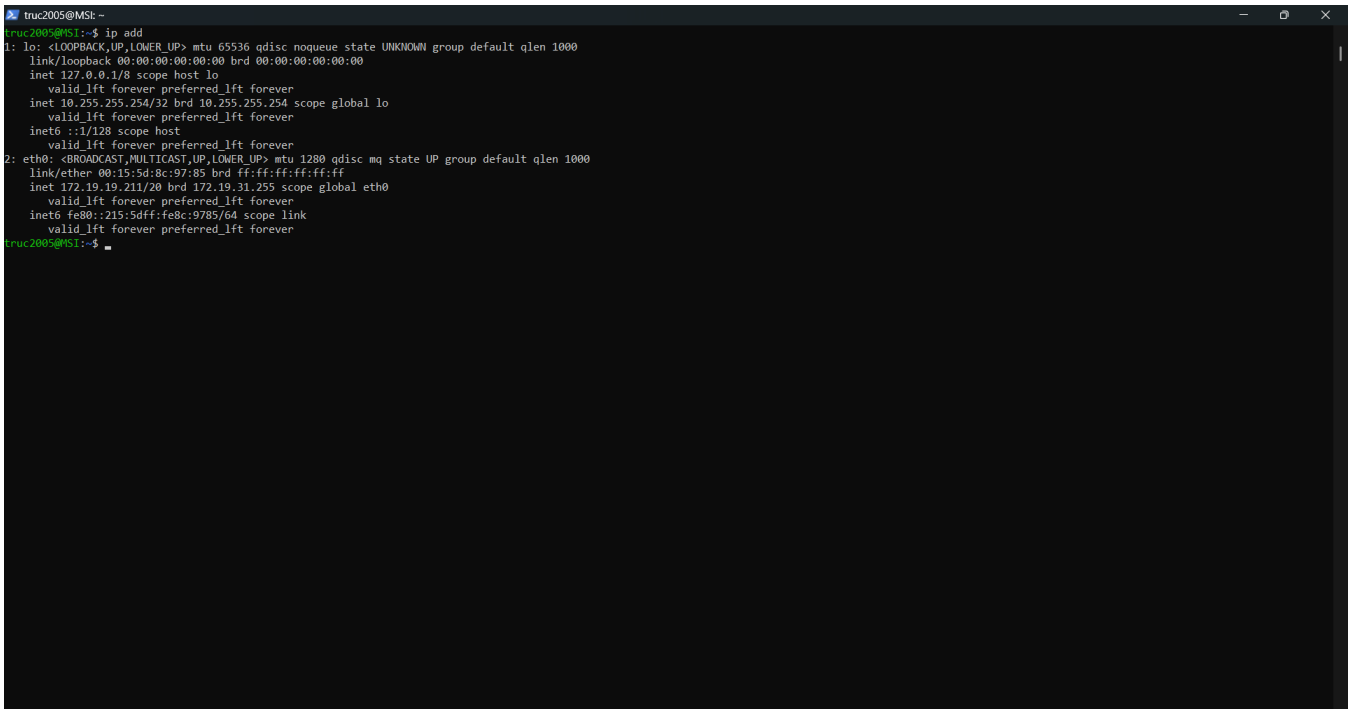
Phần này trình bày toàn bộ quá trình tôi cài đặt và cấu hình Hadoop 3.3.6 ở chế độ pseudo-distributed trên máy cá nhân chạy Windows 11, sử dụng Windows Subsystem for Linux 2 (WSL2) với Ubuntu 22.04. Mỗi bước đều được ghi lại kèm lệnh thực thi và ảnh chụp màn hình minh hoạ.

1.1 Môi trường hệ thống và địa chỉ IP

Tôi sử dụng Windows 11 kết hợp với WSL2 chạy Ubuntu 22.04 LTS. WSL2 được chọn vì Hadoop được thiết kế để chạy trên môi trường Linux, trong khi WSL2 cung cấp một nhân Linux thực sự tích hợp vào Windows mà không cần cài đặt máy ảo riêng biệt.

Ngay khi bắt đầu, tôi chạy lệnh `ip addr` để xác minh địa chỉ IP của máy trong môi trường WSL và gửi cho Leader nhóm để đối chiếu, tránh trùng lặp IP giữa các thành viên.

```
ip addr
```



Hình 1: Địa chỉ IP của máy trong WSL Ubuntu 22.04

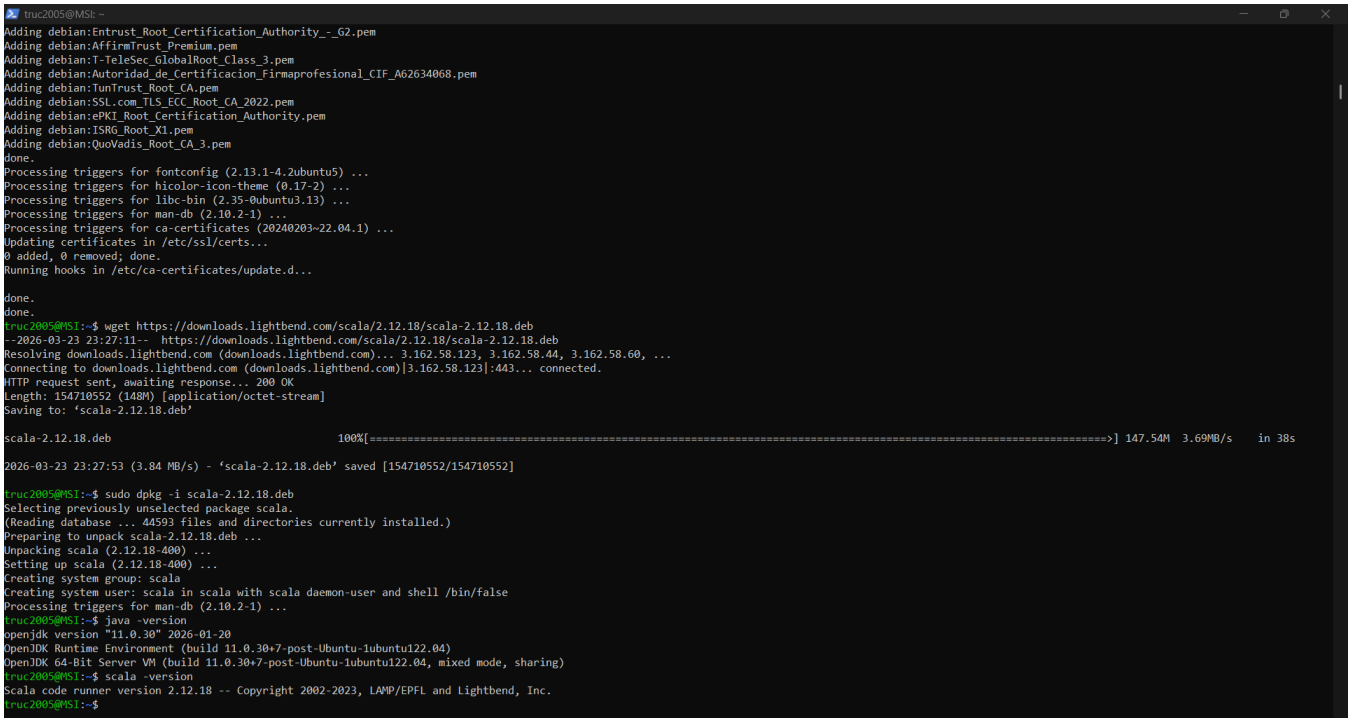
1.2 Cài đặt Java và Scala

Hadoop yêu cầu Java Runtime Environment để vận hành vì toàn bộ hệ sinh thái Hadoop được viết bằng Java. Tôi chọn OpenJDK 11 vì đây là phiên bản LTS được Hadoop 3.3.6 hỗ trợ chính thức. Scala được cài đặt để phục vụ cho việc lập trình WordCount ở phần sau.

```
sudo apt update && sudo apt upgrade -y
sudo apt install openjdk-11-jdk -y

wget https://downloads.lightbend.com/scala/2.12.18/scala-2.12.18.deb
sudo dpkg -i scala-2.12.18.deb

java -version
scala -version
```



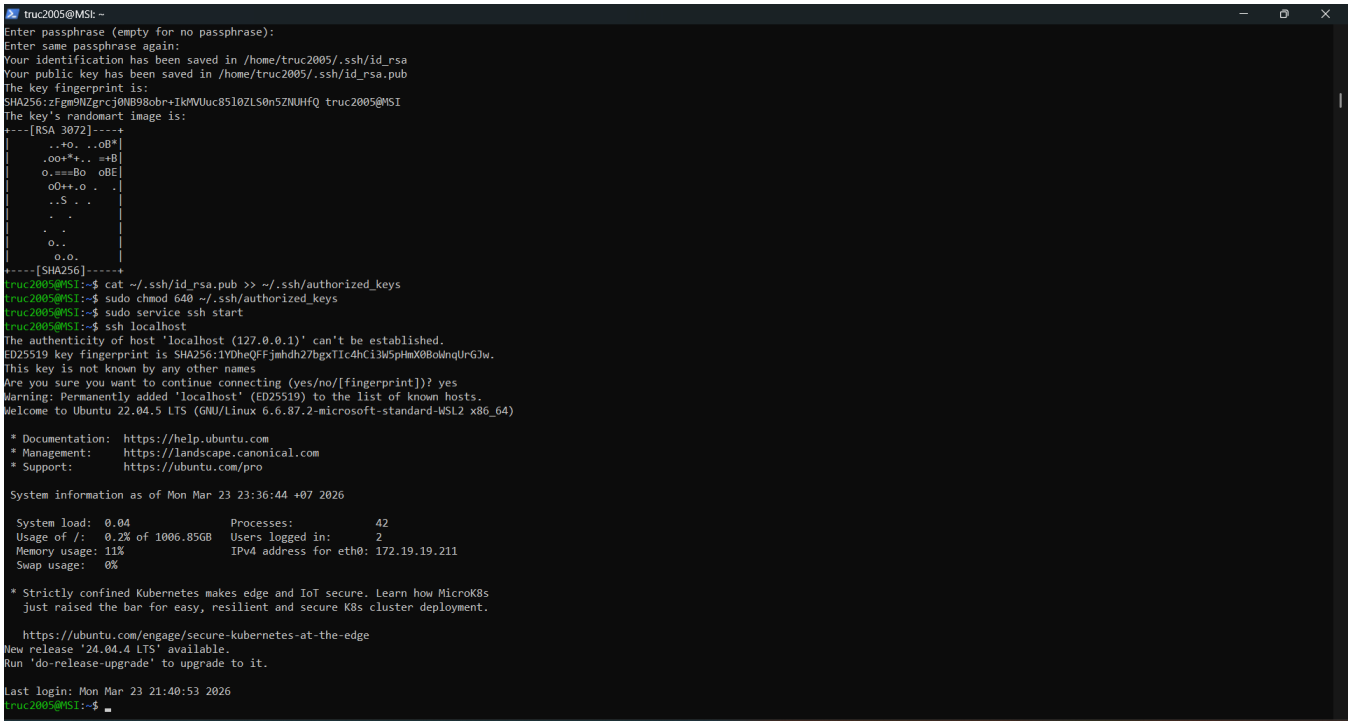
Hình 2: Cài đặt Java 11 và Scala 2.12.18 thành công

1.3 Cài đặt SSH

Hadoop sử dụng SSH để khởi động các tiến trình nội bộ - NameNode tự SSH vào localhost để khởi động DataNode. Nếu thiếu SSH, lệnh `start-dfs.sh` sẽ báo lỗi kết nối. Tôi cấu hình SSH không

mật khẩu (passwordless SSH) để Hadoop có thể tự động kết nối mà không cần nhập mật khẩu mỗi lần.

```
sudo apt install ssh -y
ssh-keygen -t rsa
cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys
sudo chmod 640 ~/.ssh/authorized_keys
sudo service ssh start
ssh localhost
```

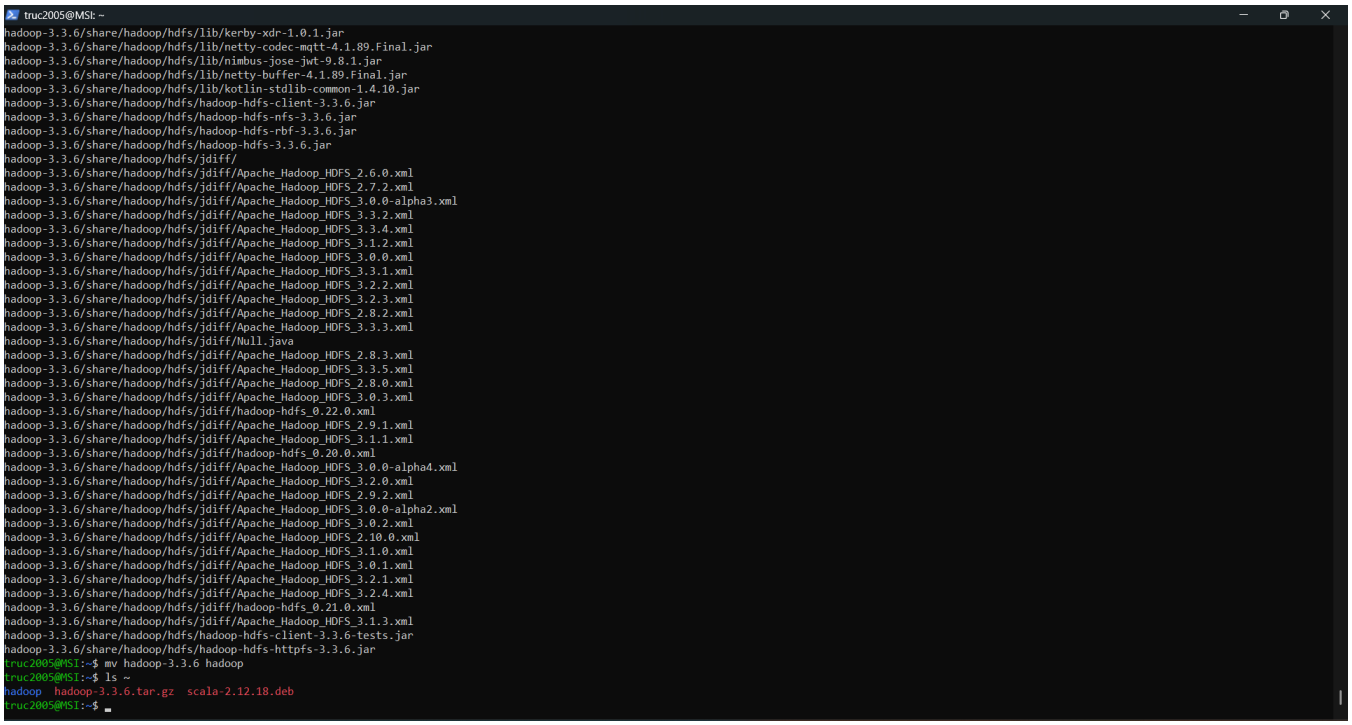


Hình 3: Cấu hình SSH thành công - kết nối localhost không cần mật khẩu

1.4 Tải và giải nén Hadoop

Tôi tải Hadoop 3.3.6 trực tiếp từ máy chủ phân phối chính thức của Apache bằng lệnh `wget`. Sau khi tải về, file nén `.tar.gz` được giải nén vào thư mục home và đổi tên thành `hadoop`.

```
cd ~
wget https://dlcdn.apache.org/hadoop/common/hadoop-3.3.6/hadoop-3.3.6.tar.gz
tar -xvzf hadoop-3.3.6.tar.gz
mv hadoop-3.3.6 hadoop
ls ~
```



Hình 4: Tải và kiểm tra bộ cài Hadoop 3.3.6

1.5 Cấu hình biến môi trường

Các biến môi trường cần thiết để hệ thống và Hadoop tìm đúng đường dẫn đến Java và các công cụ Hadoop. Biến `JAVA_HOME` chỉ đường dẫn đến thư mục cài đặt JDK, `HADOOP_HOME` trỏ đến thư mục Hadoop đã giải nén, còn biến `PATH` được bổ sung để có thể gọi trực tiếp các lệnh như `hdfs`, `start-dfs.sh` từ bất kỳ đâu trong terminal. Tôi thêm các dòng này vào cuối file `~/.bashrc` để tự động nạp khi mở terminal mới.

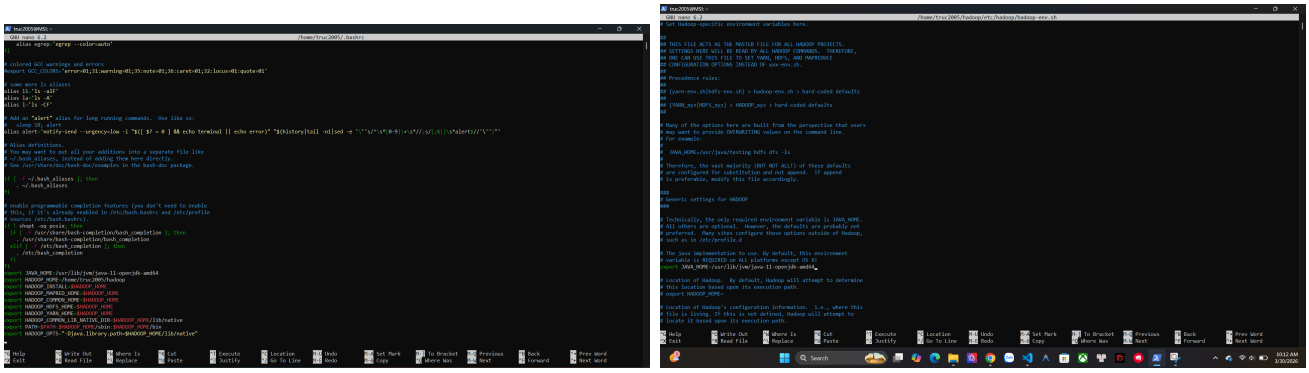
```
nano ~/.bashrc
# Them vao cuoi file:
```

```
export JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64
export HADOOP_HOME=/home/truc2005/hadoop
export HADOOP_INSTALL=$HADOOP_HOME
export HADOOP_MAPRED_HOME=$HADOOP_HOME
export HADOOP_COMMON_HOME=$HADOOP_HOME
export HADOOP_HDFS_HOME=$HADOOP_HOME
export HADOOP_YARN_HOME=$HADOOP_HOME
export HADOOP_COMMON_LIB_NATIVE_DIR=$HADOOP_HOME/lib/native
export PATH=$PATH:$HADOOP_HOME/sbin:$HADOOP_HOME/bin
```

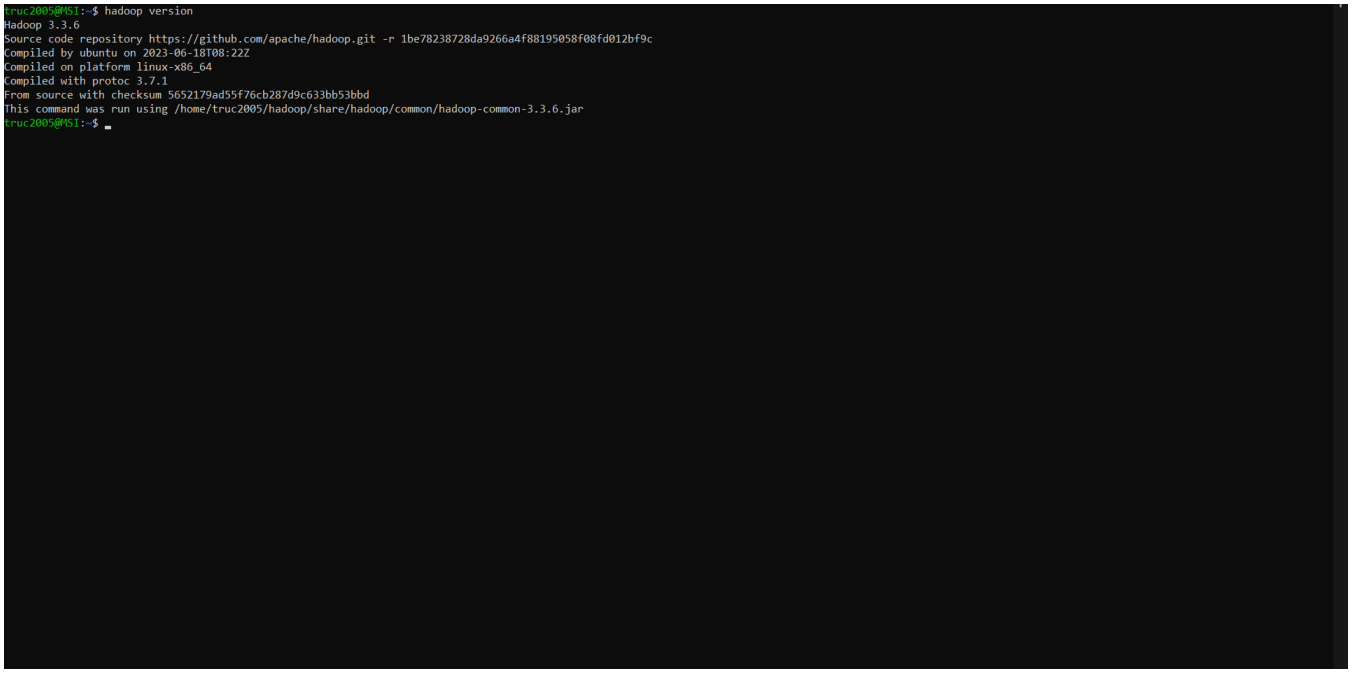
```
export HADOOP_OPTS="-Djava.library.path=$HADOOP_HOME/lib/native"
```

Ngoài ra, tôi cũng khai báo `JAVA_HOME` trực tiếp trong file `hadoop-env.sh` vì các script daemon của Hadoop không đọc `.bashrc` khi khởi động:

```
nano $HADOOP_HOME/etc/hadoop/hadoop-env.sh
# Tim va sua dong:
export JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64
```



Hình 5: Cấu hình biến môi trường trong /.bashrc và hadoop-env.sh



Hình 6: Lệnh hadoop version trả về Hadoop 3.3.6

1.6 Cấu hình XML (core-site.xml, hdfs-site.xml)

Hadoop sử dụng các file XML để cấu hình hành vi của từng thành phần. File `core-site.xml` xác định địa chỉ mặc định của HDFS thông qua thuộc tính `fs.defaultFS`; ở chế độ pseudo-distributed, giá trị là `hdfs://localhost:9000`. File `hdfs-site.xml` cấu hình các thông số của HDFS: `dfs.replication` đặt bằng 1 vì chỉ có một node, `dfs.name.dir` chỉ đường dẫn lưu metadata của NameNode, và `dfs.data.dir` chỉ đường dẫn lưu dữ liệu thực tế của DataNode. Tôi tạo trước các thư mục lưu trữ trên máy thật trước khi cấu hình:

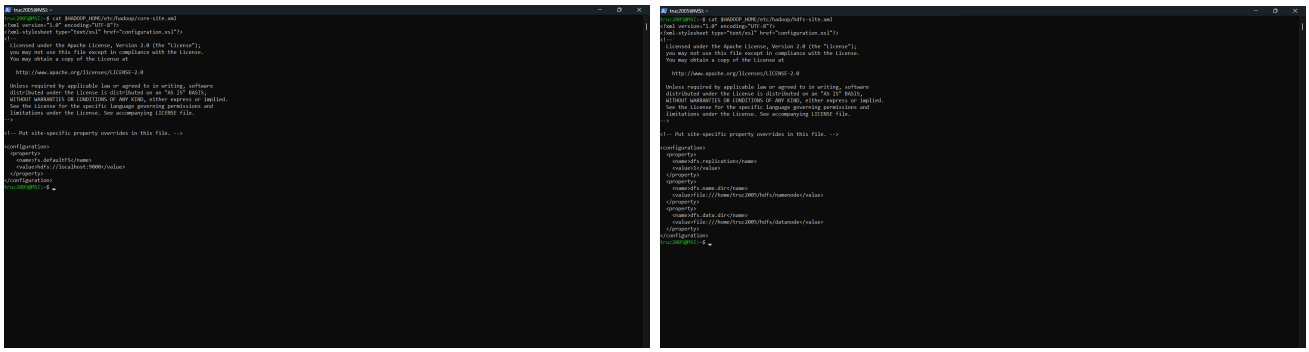
```
mkdir -p ~/hdfs/namenode
mkdir -p ~/hdfs/datanode
```

Nội dung file `core-site.xml`:

```
<configuration>
  <property>
    <name>fs.defaultFS</name>
    <value>hdfs://localhost:9000</value>
  </property>
</configuration>
```

Nội dung file `hdfs-site.xml`:

```
<configuration>
  <property>
    <name>dfs.replication</name>
    <value>1</value>
  </property>
  <property>
    <name>dfs.namenode.name.dir</name>
    <value>file:///home/truc2005/hdfs/namenode</value>
  </property>
  <property>
    <name>dfs.datanode.data.dir</name>
    <value>file:///home/truc2005/hdfs/datanode</value>
  </property>
</configuration>
```

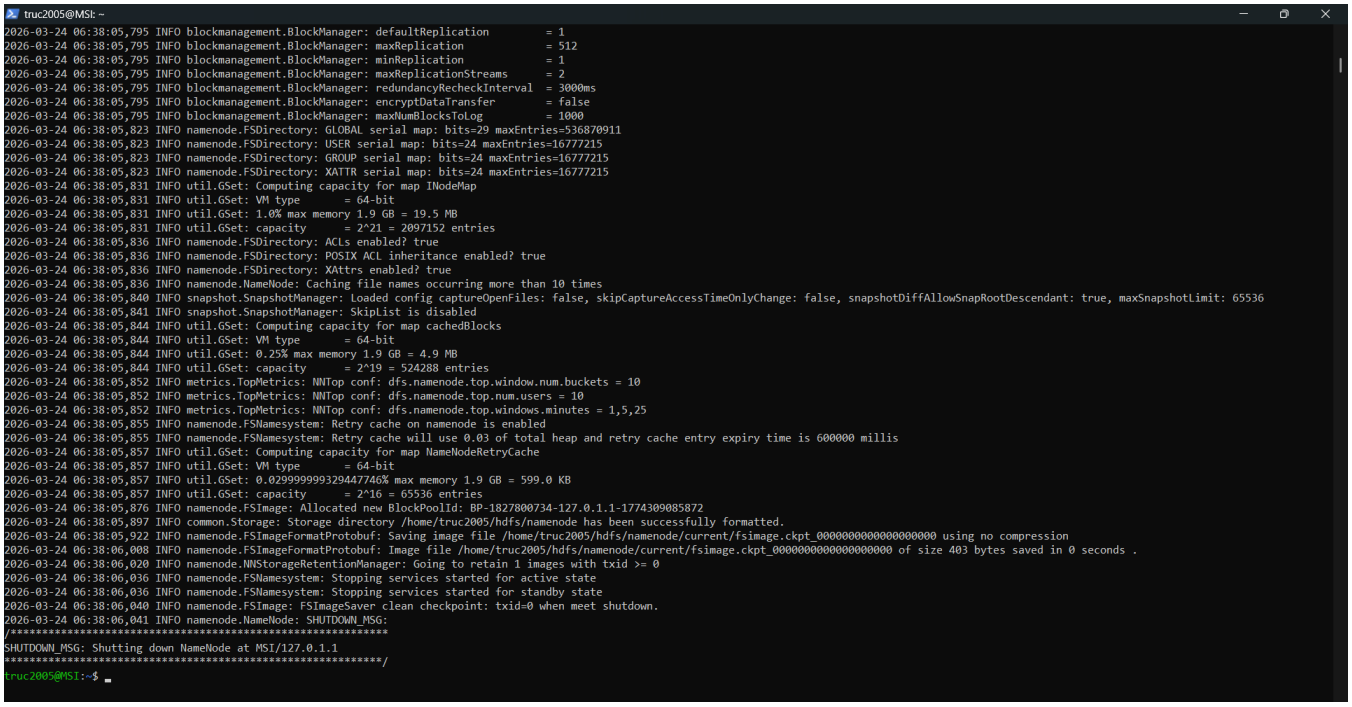


Hình 7: Nội dung cấu hình core-site.xml và hdfs-site.xml

1.7 Format NameNode

Trước khi khởi động HDFS lần đầu tiên, NameNode cần được format để khởi tạo cấu trúc thư mục metadata và tạo ID định danh cho cluster. Quá trình này tương tự như định dạng một ổ đĩa cứng mới. **Lưu ý quan trọng:** không nên chạy lệnh format lại sau khi cluster đã có dữ liệu vì sẽ khiến DataNode không nhận ra cluster ID mới và toàn bộ dữ liệu trên HDFS sẽ không còn truy cập được. Lệnh format chỉ thực hiện một lần duy nhất khi cài đặt mới.

```
hdfs namenode -format
```

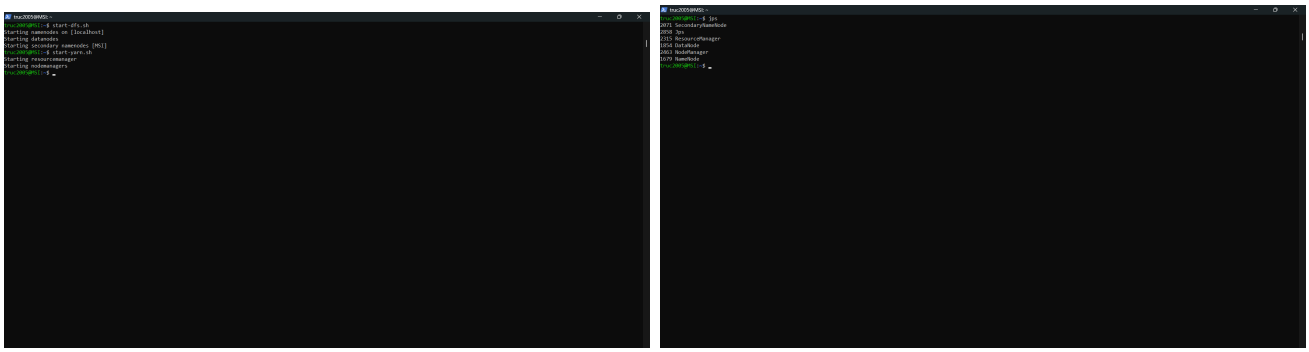


Hình 8: Format NameNode thành công - thông báo “successfully formatted”

1.8 Khởi động Cluster và kiểm tra tiến trình

Sau khi format xong, tôi tiến hành khởi động cluster bằng hai script `start-dfs.sh` và `start-yarn.sh`. Sau đó dùng lệnh `jps` để xác nhận đủ 5 tiến trình daemon: NameNode, DataNode, SecondaryNameNode, ResourceManager và NodeManager. Nếu thiếu bất kỳ tiến trình nào, cluster chưa hoạt động đúng và cần kiểm tra log.

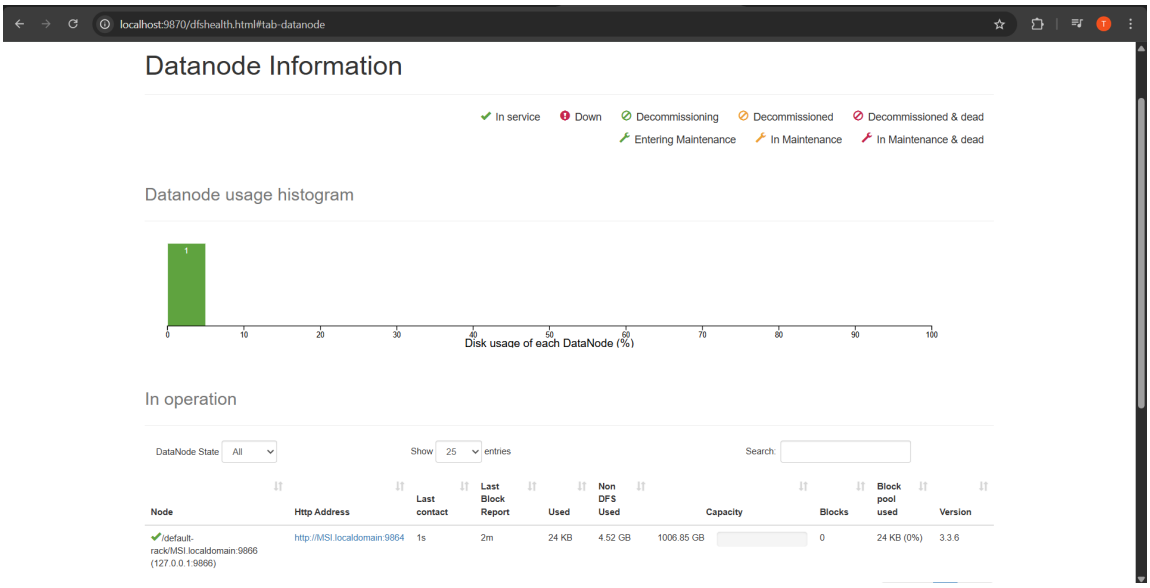
```
sudo service ssh start
start-dfs.sh
start-yarn.sh
jps
```



Hình 9: Khởi động DFS, YARN và kiểm tra tiến trình JPS đủ 5 daemon

1.9 Kiểm tra WebUI

Hadoop cung cấp giao diện web để giám sát trạng thái cluster. Tôi truy cập `http://localhost:9870` trên trình duyệt Windows. Tab *Datanodes* hiển thị trạng thái *In service* với 1 DataNode đang hoạt động, xác nhận DataNode đã kết nối thành công với NameNode.

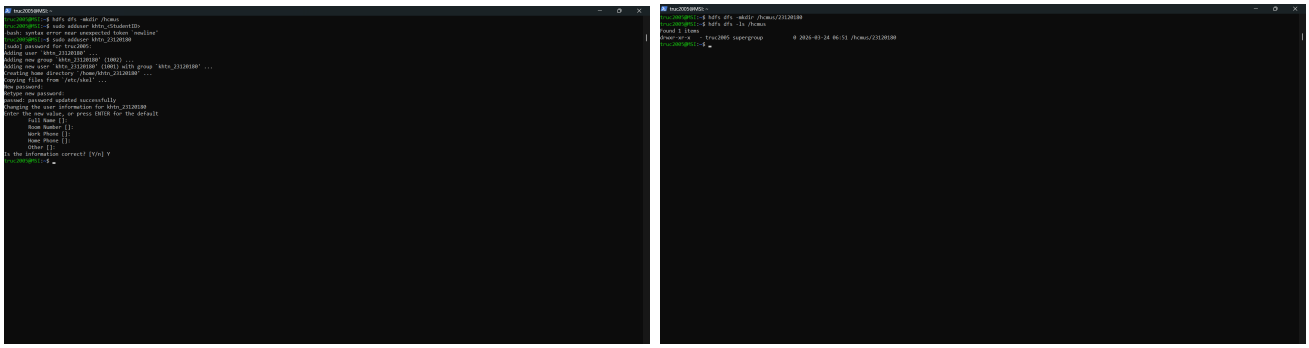


Hình 10: WebUI Hadoop tại localhost:9870 - tab Datanodes hiển thị 1 node In service

1.10 Tạo thư mục HDFS và User Linux

Theo yêu cầu bài lab, tôi tạo thư mục gốc `/hcmus` trên HDFS để chứa dữ liệu của tất cả thành viên, sau đó tạo thư mục con `/hcmus/23120180` cho MSSV của tôi. Song song, tôi tạo tài khoản Linux `khtn_23120180` để phục vụ bước phân quyền ở mục tiếp theo; tài khoản này sẽ được gán làm chủ sở hữu của thư mục HDFS tương ứng.

```
hdfs dfs -mkdir /hcmus
sudo adduser khtn_23120180
hdfs dfs -mkdir /hcmus/23120180
hdfs dfs -ls /hcmus
```

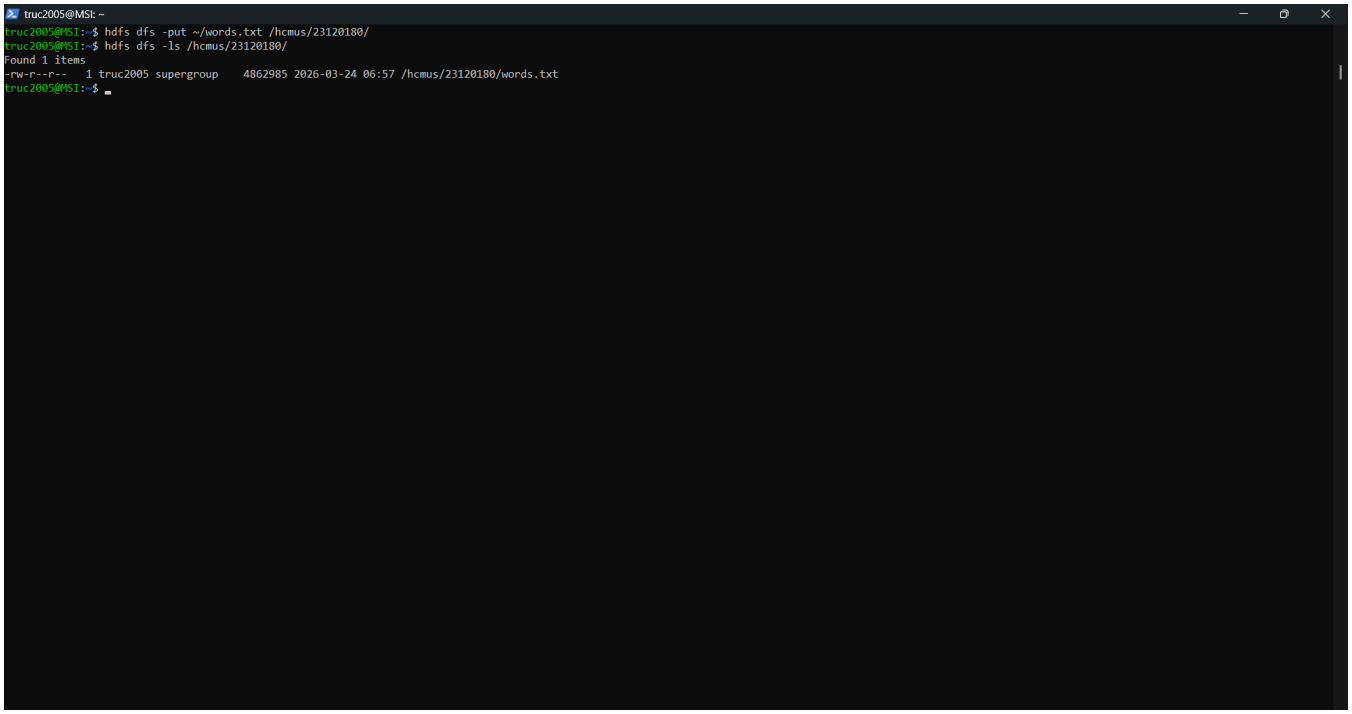


Hình 11: Tạo thư mục HDFS /hcmus/23120180 và user Linux khtn_23120180

1.11 Tải dữ liệu lên HDFS

Tôi tải file `words.txt` lên thư mục HDFS đã tạo bằng lệnh `hdfs dfs -put`. File được copy từ Windows Downloads vào WSL trước, sau đó upload lên HDFS. Sau khi tải lên, tôi xác nhận file đã tồn tại bằng lệnh `-ls`.

```
cp /mnt/c/Users/truct/Downloads/words.txt ~/
hdfs dfs -put ~/words.txt /hcmus/23120180/
hdfs dfs -ls /hcmus/23120180/
```



Hình 12: Tải file words.txt lên HDFS và xác nhận bằng lệnh ls

1.12 Phân quyền và chuyển Owner

Tôi thực hiện phân quyền theo chuẩn Unix octal `744` - nghĩa là chủ sở hữu có toàn quyền đọc-ghi-thực thi (`rwx`), còn nhóm và người khác chỉ có quyền đọc (`r-`) - và chuyển quyền sở hữu thư mục sang tài khoản `khtn_23120180`.

```
hdfs dfs -chmod 744 /hcmus/23120180
hdfs dfs -chown khtn_23120180 /hcmus/23120180
hdfs dfs -ls /hcmus/
```

Lỗi gặp phải - JAR báo "Ensure the permission is set to 744":

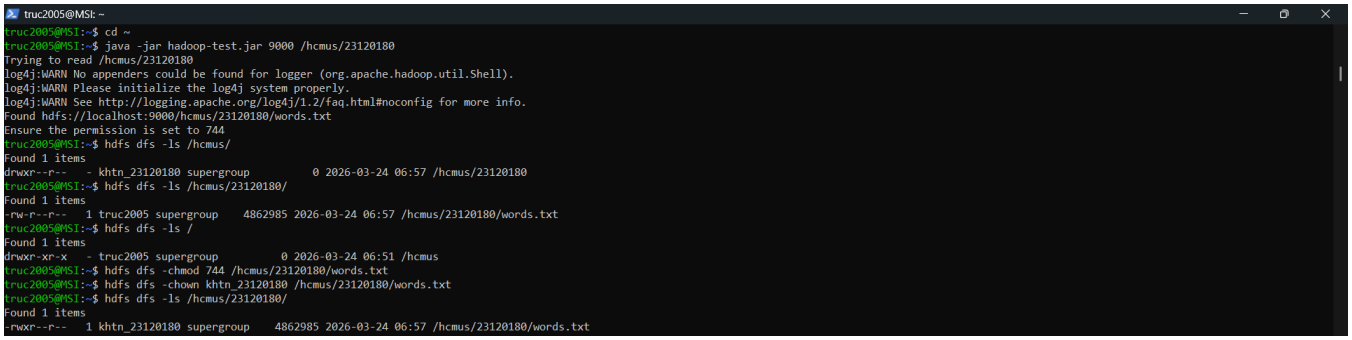
Sau khi thực hiện chmod và chown cho thư mục `/hcmus/23120180`, tôi chạy JAR xác thực nhưng vẫn nhận được thông báo:

```
Found hdfs://localhost:9000/hcmus/23120180/words.txt
Ensure the permission is set to 744
```

Tôi kiểm tra lại bằng lệnh `hdfs dfs -ls /hcmus/23120180/` và phát hiện file `words.txt` bên trong vẫn có owner là `truc2005` và permission mặc định `rw-r-r-`. JAR không chỉ kiểm tra thư mục mà còn kiểm tra cả file bên trong. Tôi sửa bằng cách chmod và chown thêm cho file:

```
hdfs dfs -chmod 744 /hcmus/23120180/words.txt
hdfs dfs -chown khtn_23120180 /hcmus/23120180/words.txt
hdfs dfs -ls /hcmus/23120180/
```

Sau khi thực hiện đầy đủ, chạy lại JAR cho kết quả thành công.



Hình 13: Phân quyền chmod 744 và chuyển owner cho thư mục và file bên trong

1.13 Chạy file JAR xác thực

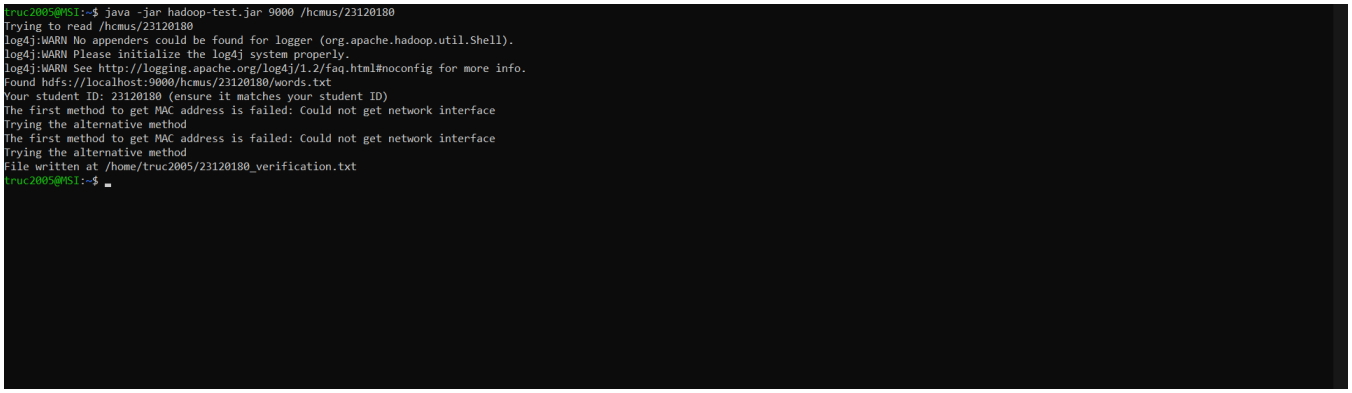
File `hadoop-test.jar` là công cụ xác thực do giáo viên cung cấp, có nhiệm vụ kết nối vào HDFS, đọc thông tin thư mục của sinh viên, ghi nhận thông tin mạng của máy, và tạo ra file kết quả `23120180_verification.txt` chứa thông tin xác thực duy nhất. File này dùng để chứng minh sinh viên đã thực sự cài đặt và vận hành Hadoop thành công trên máy của mình.

```
cp /mnt/c/Users/truct/Downloads/hadoop-test.jar ~/
cd ~
java -jar hadoop-test.jar 9000 /hcmus/23120180
cat ~/23120180_verification.txt
```

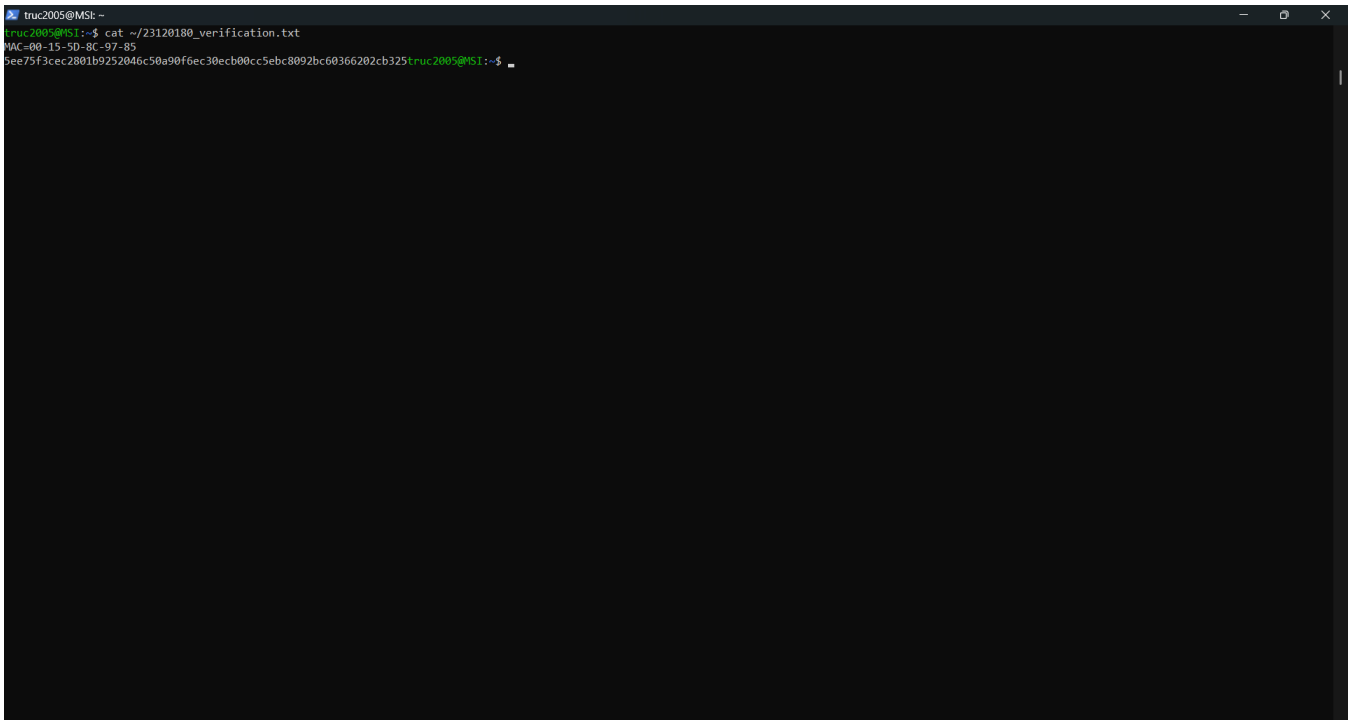
Output thu được khi chạy JAR thành công:

```
Trying to read /hcmus/23120180
```

```
Found hdfs://localhost:9000/hcmus/23120180/words.txt
Your student ID: 23120180 (ensure it matches your student ID)
The first method to get MAC address is failed: Could not get network interface
Trying the alternative method
File written at /home/truc2005/23120180_verification.txt
```



Hình 14: Chạy hadoop-test.jar thành công và tạo file xác thực



Hình 15: Nội dung file 23120180_verification.txt

2 WordCount bằng Scala

2.1 Giới thiệu bài toán

Bài toán WordCount yêu cầu đếm số lượng từ trong file `words.txt` theo chữ cái đầu tiên, cụ thể là 8 chữ cái: **f, i, t, h, c, m, u, s**. Chương trình cần thỏa mãn các ràng buộc sau:

- **Đầu vào:** file `words.txt` đọc trực tiếp trên HDFS tại `hdfs://localhost:9000/hcmus/23120180/words`
- **Xử lý chuỗi:** Phân tách từ bằng dấu cách đơn thuần, tuyệt đối không dùng biểu thức chính quy `regex`; chuyển tất cả về chữ thường (case-insensitive) trước khi đếm.
- **Đầu ra:** file `results.txt` định dạng TSV (ngăn cách bằng tab `\t`), 8 dòng tương ứng 8 chữ cái, sắp xếp theo thứ tự **f, i, t, h, c, m, u, s**.
- **Ngôn ngữ:** Bắt buộc sử dụng Scala.

2.2 Bước 1: Cài đặt SBT và cấu hình build.sbt

SBT (Scala Build Tool) là công cụ biên dịch và đóng gói tiêu chuẩn cho các dự án Scala. Tôi cài đặt SBT trên môi trường Ubuntu WSL2 theo cách chính thức từ repository của Lightbend:

```
sudo apt-get update
sudo apt-get install apt-transport-https curl gnupg -yqq

echo "deb https://repo.scala-sbt.org/scalasbt/debian all main" | \
  sudo tee /etc/apt/sources.list.d/sbt.list
echo "deb https://repo.scala-sbt.org/scalasbt/debian /" | \
  sudo tee /etc/apt/sources.list.d/sbt_old.list

curl -sL "https://keyserver.ubuntu.com/pks/lookup?op=get&\
search=0x2EE0EA64E40A89B84B2DF73499E82A75642AC823" | \
  sudo -H gpg --no-default-keyring \
  --keyring gnupg-ring:/etc/apt/trusted.gpg.d/scalasbt-release.gpg --import
sudo chmod 644 /etc/apt/trusted.gpg.d/scalasbt-release.gpg

sudo apt-get update
```

```
sudo apt-get install sbt -y
sbt --version
```

```
truc2005@MSI:~$ sbt --version
sbt runner version: 1.12.8

[info] sbt runner (sbt-the-shell-script) is a runner to run any declared version of sbt.
[info] Actual version of the sbt is declared using project/build.properties for each build.
```

Hình 16: Cài đặt SBT thành công, lệnh `sbt -version` trả về phiên bản đã cài

Tiếp theo, tôi tạo cấu trúc thư mục dự án chuẩn SBT và khai báo file `build.sbt` với các thư viện Hadoop cần thiết. Cờ `"provided"` báo cho SBT biết rằng các thư viện này đã có sẵn trên cụm Hadoop khi runtime, không cần đóng gói vào JAR (sẽ thay đổi ở bước sau khi gặp sự cố Fat JAR):

```
mkdir -p ~/src/WordCount/src/main/scala
nano ~/src/WordCount/build.sbt
```

```
name := "WordCountLab"
version := "1.0"
scalaVersion := "2.12.18"

libraryDependencies ++= Seq(
  "org.apache.hadoop" % "hadoop-common"              % "3.3.6" % "provided",
  "org.apache.hadoop" % "hadoop-mapreduce-client-core" % "3.3.6" % "provided"
)
```

```
truc2005@MSI:~/src/WordCount$ cat build.sbt
name := "WordCountLab"
version := "1.0"
scalaVersion := "2.12.18"

libraryDependencies ++= Seq(
  "org.apache.hadoop" % "hadoop-common" % "3.3.6" % "provided",
  "org.apache.hadoop" % "hadoop-mapreduce-client-core" % "3.3.6" % "provided"
)
```

Hình 17: Cấu hình file `build.sbt` khai báo thư viện Hadoop

2.3 Bước 2: Cấu hình YARN và MapReduce

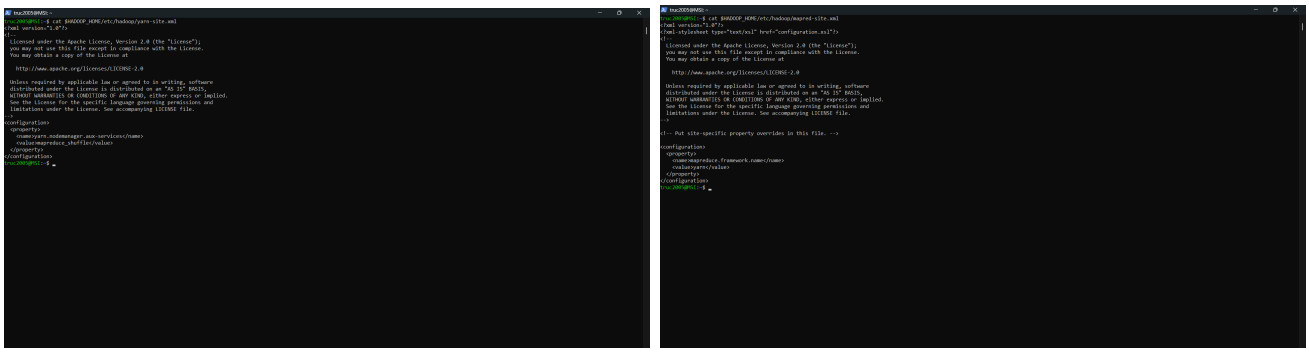
Để Hadoop biết rằng các MapReduce Job sẽ chạy trên nền tảng quản lý tài nguyên YARN (thay vì chạy cục bộ), tôi cấu hình 2 file XML quan trọng:

File yarn-site.xml - kích hoạt bộ xáo trộn dữ liệu (shuffle) của MapReduce trên các NodeManager:

```
<configuration>
  <property>
    <n>yarn.nodemanager.aux-services</n>
    <value>mapreduce_shuffle</value>
  </property>
</configuration>
```

File mapred-site.xml - chỉ định framework thực thi là YARN:

```
<configuration>
  <property>
    <n>mapreduce.framework.name</n>
    <value>yarn</value>
  </property>
</configuration>
```



Hình 18: Nội dung cấu hình `yarn-site.xml` và `mapred-site.xml`

2.4 Bước 3: Chuẩn bị dữ liệu đầu vào

File `words.txt` đã được tải lên HDFS tại mục 1.11 (Phần 1, mục 1.11).

2.5 Bước 4: Mã nguồn Scala (WordCount.scala)

Tổng quan kiến trúc

Chương trình được tổ chức theo chuẩn Hadoop MapReduce với 4 thành phần chính:

Thành phần	Lớp	Vai trò
Bộ sắp xếp tùy chỉnh	`CustomSortComparator`	Ép thứ tự đầu ra theo f,i,t,h,c,m,u,s
Bộ phân loại	`TokenizerMapper`	Đọc dữ liệu, phát cặp (chữ_cái, 1)
Bộ đếm và tổng hợp	`IntSumReducer`	Cộng dồn số đếm theo từng chữ cái
Trung tâm điều khiển	`WordCount` (object)	Khởi tạo và cấu hình toàn bộ Job

Mã nguồn đầy đủ

```scala
import java.lang.Iterable
import org.apache.hadoop.conf.Configuration
import org.apache.hadoop.fs.Path
import org.apache.hadoop.io.{IntWritable, LongWritable, Text,
  WritableComparable, WritableComparator}
import org.apache.hadoop.mapreduce.{Job, Mapper, Reducer}
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat
import scala.collection.JavaConverters._

// KHÂU 1: BỘ SẮP XẾP TÙY CHỈNH (CustomSortComparator)
// Kế thừa WritableComparator để ghi đè thuật toán Shuffle & Sort
class CustomSortComparator extends WritableComparator(classOf[Text], true) {
  // Gán "trọng số" cho từng chữ cái. Chữ có số nhỏ hơn sẽ đứng trước.
  private val order = Map(
    "f" -> 1, "i" -> 2, "t" -> 3, "h" -> 4,
    "c" -> 5, "m" -> 6, "u" -> 7, "s" -> 8
  )

  override def compare(a: WritableComparable[_],
                       b: WritableComparable[_]): Int = {
    val key1 = a.asInstanceOf[Text].toString
```

```
    val key2 = b.asInstanceOf[Text].toString
    // Chữ cái lạ (ngoài danh sách) được gán trọng số 99 - đẩy xuống cuối
    val val1 = order.getOrElse(key1, 99)
    val val2 = order.getOrElse(key2, 99)
    val1.compareTo(val2)
  }
}

// KHÂU 2: BỘ PHÂN LOẠI (Mapper)
// Đọc từng dòng văn bản, nhặt ra các từ hợp lệ và phát tín hiệu (chữ_cái, 1)
class TokenizerMapper
    extends Mapper[LongWritable, Text, Text, IntWritable] {
  private val one      = new IntWritable(1)
  private val wordKey  = new Text()
  private val targetChars = Set('f', 'i', 't', 'h', 'c', 'm', 'u', 's')

  override def map(key: LongWritable, value: Text,
      context: Mapper[LongWritable, Text, Text, IntWritable]#Context): Unit = {
    // Tách dòng thành từ bằng dấu cách đơn thuần (KHÔNG DÙNG REGEX)
    val words = value.toString.split(" ")

    for (w <- words) {
      val trimmed = w.trim
      if (trimmed.nonEmpty) {
        val lower     = trimmed.toLowerCase   // case-insensitive
        val firstChar = lower.charAt(0)
        if (targetChars.contains(firstChar)) {
          wordKey.set(firstChar.toString)
          context.write(wordKey, one)         // Ví dụ: ("f", 1)
        }
      }
    }
  }
}

// KHÂU 3: BỘ ĐẾM (Reducer)
// Gom tất cả các số 1 của cùng chữ cái lại và cộng dồn
```

```scala
class IntSumReducer
    extends Reducer[Text, IntWritable, Text, IntWritable] {
  override def reduce(key: Text, values: Iterable[IntWritable],
      context: Reducer[Text, IntWritable, Text, IntWritable]#Context): Unit = {
    var sum = 0
    for (value <- values.asScala) {
      sum += value.get()
    }
    // Hadoop tự động chèn Tab (\t) giữa key và value => định dạng TSV
    context.write(key, new IntWritable(sum))  // Ví dụ: ("f", 15870)
  }
}

// KHÂU 4: TRUNG TÂM ĐIỀU KHIỂN (Driver)
// Cấu hình và khởi động toàn bộ dây chuyền MapReduce
object WordCount {
  def main(args: Array[String]): Unit = {
    val conf = new Configuration()
    val job  = Job.getInstance(conf, "WordCount_23120180")

    job.setJarByClass(this.getClass)
    job.setMapperClass(classOf[TokenizerMapper])
    job.setCombinerClass(classOf[IntSumReducer])  // Gộp tạm ở Mapper để tối ưu
    job.setReducerClass(classOf[IntSumReducer])

    // Gắn CustomSortComparator để ép thứ tự f,i,t,h,c,m,u,s
    job.setSortComparatorClass(classOf[CustomSortComparator])

    job.setOutputKeyClass(classOf[Text])
    job.setOutputValueClass(classOf[IntWritable])

    val inputPath  = new Path(
      "hdfs://localhost:9000/hcmus/23120180/words.txt")
    val outputPath = new Path(
      "hdfs://localhost:9000/hcmus/23120180/output_wordcount")

    // An toàn: xóa thư mục output cũ nếu đã tồn tại từ lần chạy trước
```

```
    val fs = outputPath.getFileSystem(conf)
    if (fs.exists(outputPath)) { fs.delete(outputPath, true) }

    FileInputFormat.addInputPath(job, inputPath)
    FileOutputFormat.setOutputPath(job, outputPath)

    System.exit(if (job.waitForCompletion(true)) 0 else 1)
  }
}
```

Giải thích các thành phần chính

CustomSortComparator: Mặc định, Hadoop sắp xếp output theo thứ tự từ điển (a–z). Bài lab yêu cầu thứ tự tùy chỉnh **f, i, t, h, c, m, u, s**. Bằng cách kế thừa `WritableComparator` và ghi đè phương thức `compare()`, tôi gán trọng số số nguyên cho mỗi chữ cái (f=1, i=2, ..., s=8). Hadoop sẽ gọi comparator này trong giai đoạn *Shuffle & Sort* để sắp xếp các cặp key theo trọng số thay vì theo bảng chữ cái.

TokenizerMapper: Mỗi dòng văn bản đầu vào được tách thành các token bằng `split(" ")` (ký tự khoảng trắng đơn, không dùng regex). Từng token sau khi `trim()` và `toLowerCase()` sẽ được kiểm tra ký tự đầu có thuộc tập `targetChars` không. Nếu có, phát ra cặp `(chữ_cái, 1)` vào luồng trung gian của Hadoop.

IntSumReducer: Đảm nhận đồng thời vai trò *Combiner* (gộp tạm ngay tại Mapper node để giảm lưu lượng mạng) và *Reducer* chính thức (tổng hợp kết quả cuối). Với mỗi key (chữ cái), Reducer nhận một danh sách `[1, 1, 1, ...]` và cộng dồn thành tổng cuối. Hadoop tự động định dạng output thành TSV bằng cách chèn ký tự Tab giữa key và value.

Driver (object WordCount): Khởi tạo `Job`, kết nối 4 thành phần lại, đặt đường dẫn HDFS cho input và output. Đoạn logic tự xóa thư mục output cũ (`fs.delete`) giúp tránh lỗi `FileAlreadyExistsException` khi chạy lại nhiều lần.

2.6 Bước 5: Biên dịch và xử lý sự cố

Sự cố 1: Lỗi thiếu thư viện Scala (`NoClassDefFoundError`)

Ban đầu, tôi dùng lệnh `sbt package` để đóng gói và gửi JAR lên Hadoop. Hệ thống văng lỗi:

```
java.lang.NoClassDefFoundError: scala/runtime/BoxedUnit
```

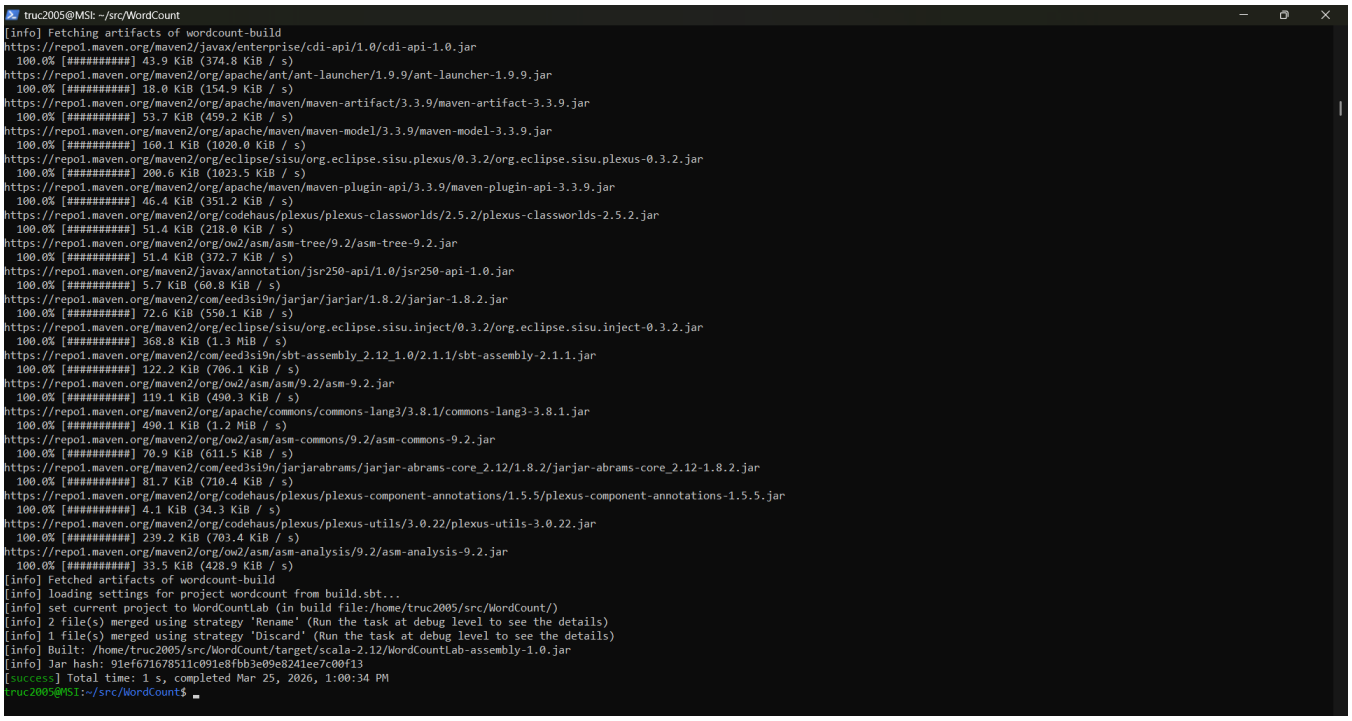
Nguyên nhân: Cụm Hadoop chạy nền Java thuần và không được cài sẵn thư viện lõi Scala. JAR được tạo bởi `sbt package` chỉ chứa bytecode dự án, không kèm thư viện Scala runtime.

Giải pháp: Dùng plugin `sbt-assembly` để tạo *Fat JAR* - một file JAR duy nhất chứa toàn bộ mã nguồn **và** tất cả thư viện phụ thuộc. Tôi tạo file `project/plugins.sbt`:

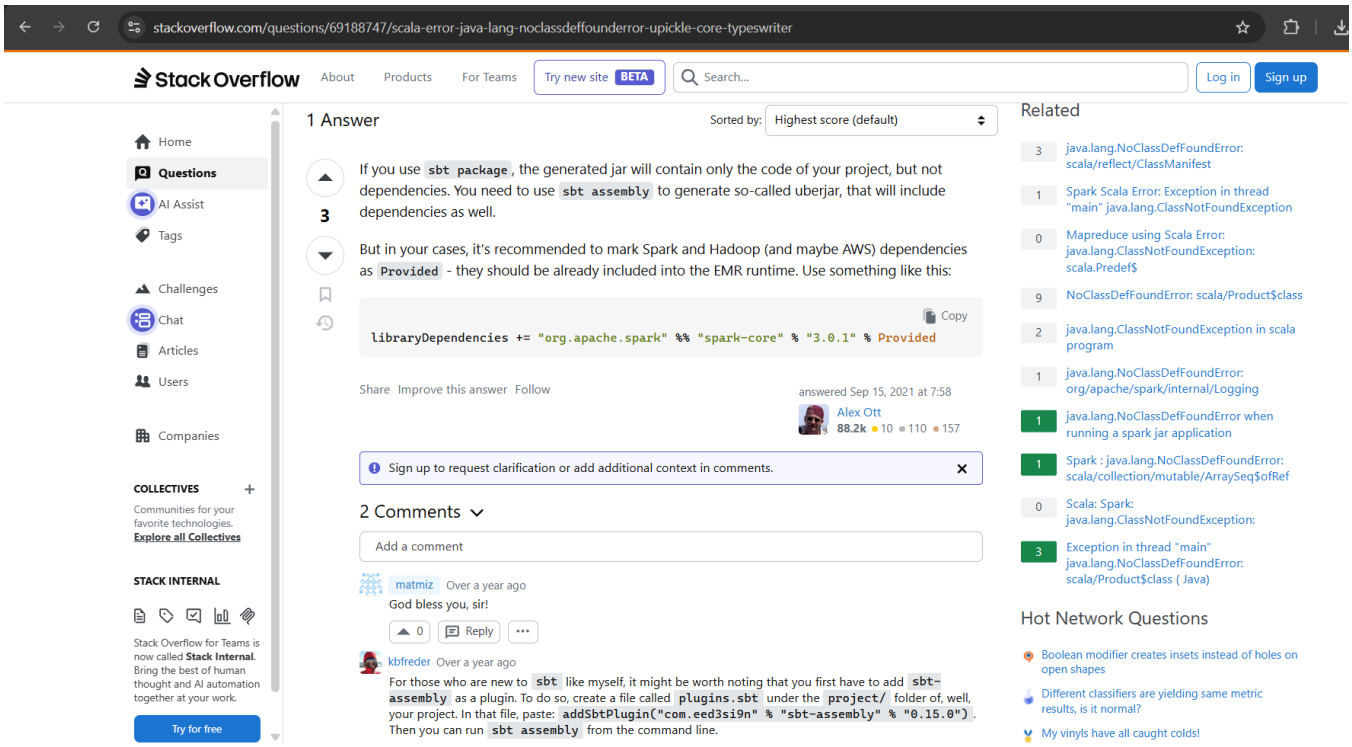
```
addSbtPlugin("com.eed3si9n" % "sbt-assembly" % "2.1.1")
```

Sau đó biên dịch lại:

```
sbt assembly
```



Hình 19: Biên dịch Fat JAR bằng `sbt assembly` thành công

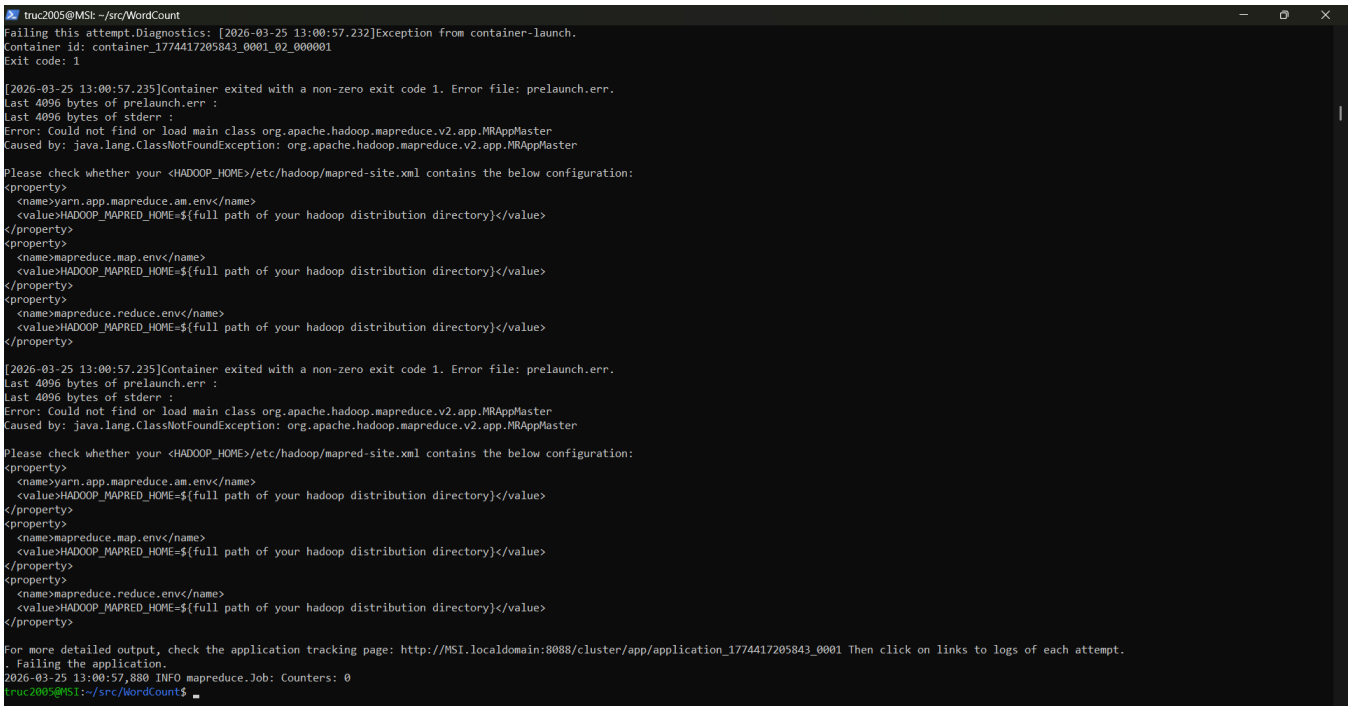


Hình 20: Quá trình tra cứu và giải quyết lỗi NoClassDefFoundError trên diễn đàn StackOverflow

Sự cố 2: Lỗi YARN Container crash (`MRAppMaster`)

Khi gửi Fat JAR lên cụm bằng lệnh `hadoop jar`, YARN Container bị crash:

```
Could not find or load main class
    org.apache.hadoop.mapreduce.v2.app.MRAppMaster
```

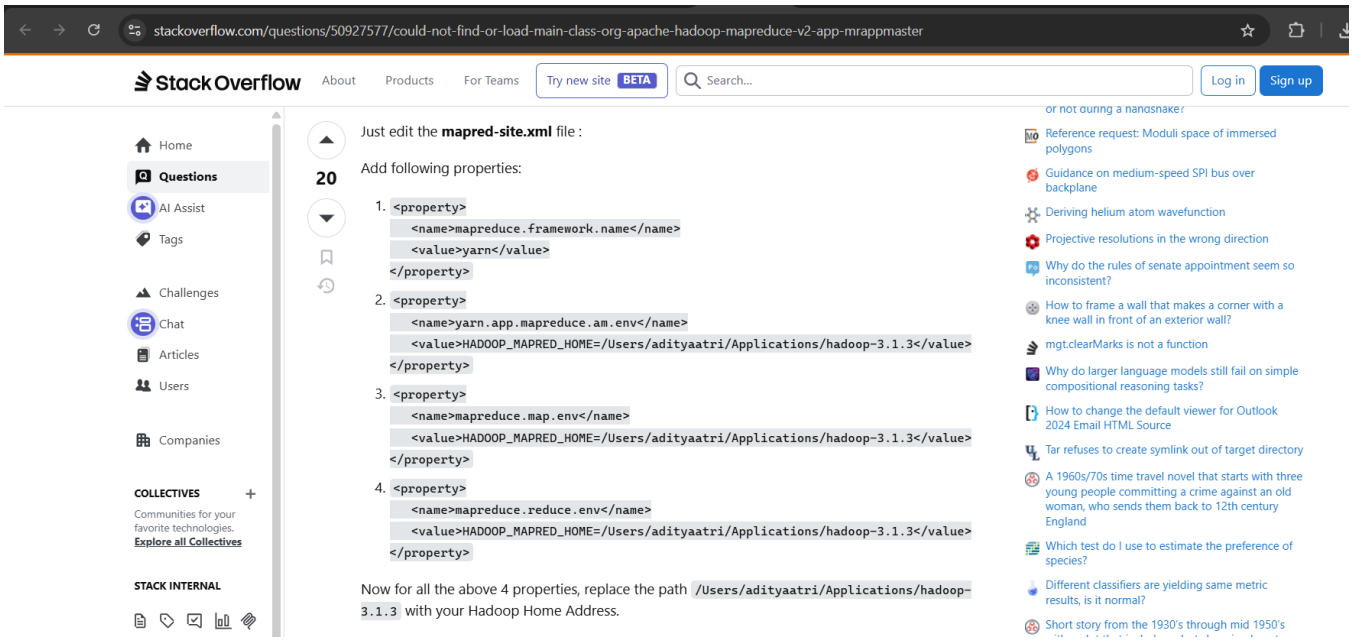


Hình 21: Lỗi `MRAppMaster`: YARN ApplicationMaster không tìm thấy classpath của MapReduce

Nguyên nhân: YARN ApplicationMaster không tìm được đường dẫn đến các JAR của hệ sinh thái MapReduce vì biến môi trường `HADOOP_MAPRED_HOME` chưa được khai báo tường minh trong cấu hình YARN.

Giải pháp: Tra cứu cộng đồng StackOverflow xác nhận cần bổ sung 3 thuộc tính môi trường vào `mapred-site.xml`, trỏ về thư mục cài đặt Hadoop:

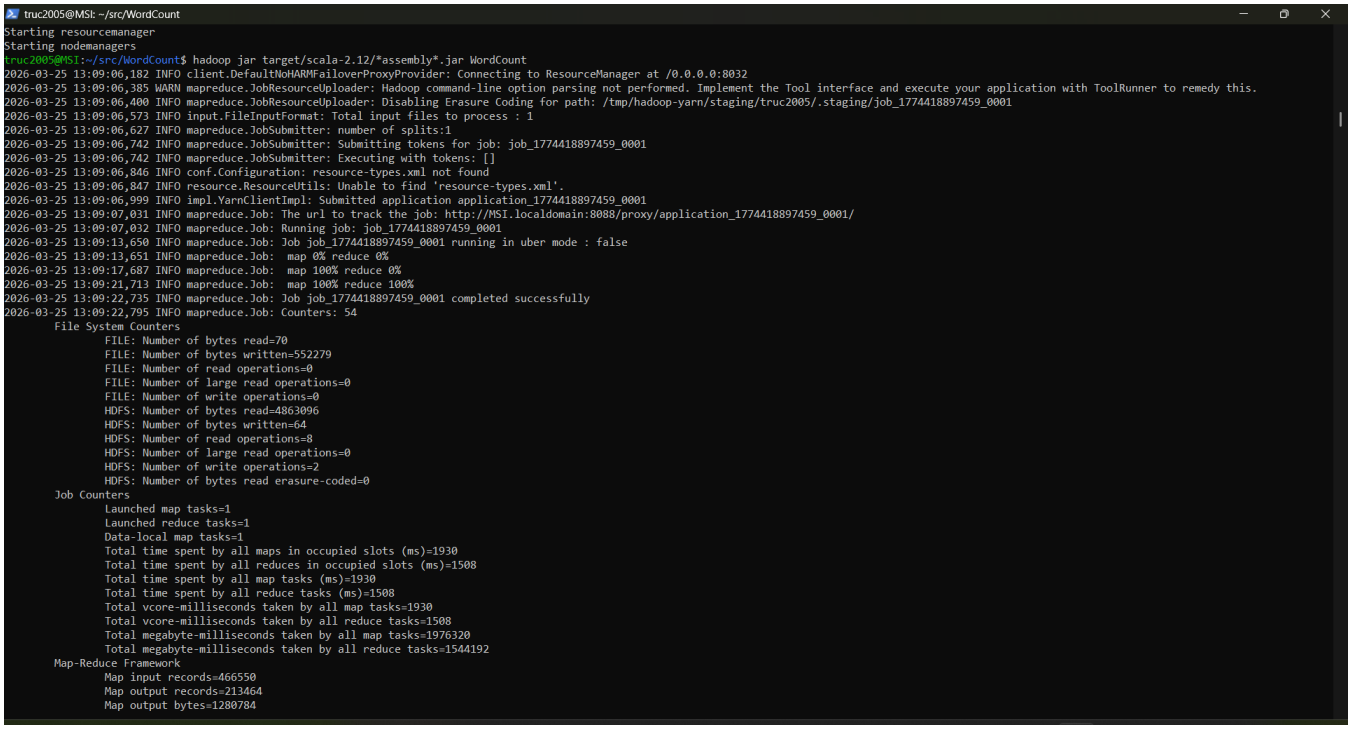
```
<property>
  <n>yarn.app.mapreduce.am.env</n>
  <value>HADOOP_MAPRED_HOME=/home/truc2005/hadoop</value>
</property>
<property>
  <n>mapreduce.map.env</n>
  <value>HADOOP_MAPRED_HOME=/home/truc2005/hadoop</value>
</property>
<property>
  <n>mapreduce.reduce.env</n>
  <value>HADOOP_MAPRED_HOME=/home/truc2005/hadoop</value>
</property>
```



Hình 22: Quá trình tra cứu và giải quyết lỗi MRAppMaster trên diễn đàn StackOverflow

Sau khi khởi động lại YARN và gửi lại Job:

```
stop-yarn.sh && start-yarn.sh
hadoop jar target/scala-2.12/*assembly*.jar WordCount
```



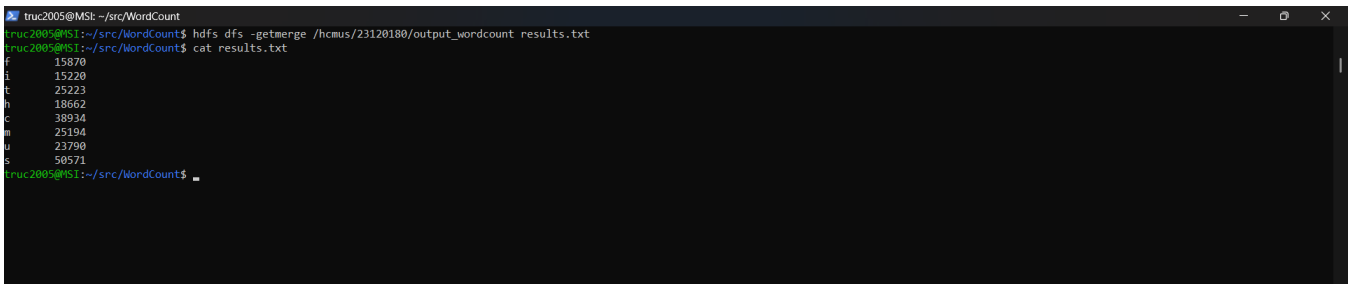
Hình 23: Job MapReduce hoàn thành thành công: `map 100% reduce 100%`

2.7 Bước 6: Trích xuất và kiểm tra kết quả

Vì MapReduce phân tán output ra nhiều phân vùng (partitions), tôi dùng lệnh `-getmerge` để gom toàn bộ về một file duy nhất:

```
hdfs dfs -getmerge /hcmus/23120180/output_wordcount results.txt
cat results.txt
```

Kết quả hiển thị chính xác định dạng TSV và tuân thủ thứ tự **f, i, t, h, c, m, u, s** nhờ `CustomSortComparator`:



Hình 24: Nội dung file `results.txt` - định dạng TSV, sắp xếp đúng thứ tự f, i, t, h, c, m, u, s

3 Cài đặt Cụm Hadoop Fully-Distributed

Để xây dựng cụm Hadoop fully-distributed mà không cần nhiều máy vật lý, tôi sử dụng **Docker** để giả lập môi trường đa node ngay trên một máy tính duy nhất. Mỗi container Docker đóng vai trò như một máy chủ riêng biệt với hostname độc lập, tạo thành cụm gồm 1 Master và 2 Worker kết nối qua mạng nội bộ ảo.

3.1 Giai đoạn 1: Chuẩn bị môi trường Docker

Bước 1: Tạo mạng ảo nội bộ (Docker Bridge Network)

Mạng Bridge cho phép các container tự động nhận diện nhau qua tên hostname mà không cần cấu hình địa chỉ IP tĩnh:

```
docker network create --driver bridge hadoop_net
```

Bước 2: Khởi tạo 3 Container (1 Master, 2 Worker)

```
docker run -itd --net=hadoop_net --name master --hostname master ubuntu:20.04
docker run -itd --net=hadoop_net --name worker1 --hostname worker1 ubuntu:20.04
docker run -itd --net=hadoop_net --name worker2 --hostname worker2 ubuntu:20.04
```

Hình 25: Khởi tạo 3 container Docker thành công

3.2 Giai đoạn 2: Cấu hình SSH không mật khẩu

Để Master có thể tự động điều khiển các Worker, cần thiết lập SSH không mật khẩu (passwordless SSH). Tôi thực hiện các bước sau:

Bước 1: Cài đặt SSH server và client trên cả 3 container.

Bước 2: Tạo cặp khóa RSA trên Master và phân phối public key sang các Worker:

```
docker exec master ssh-keygen -t rsa -P "" -f /root/.ssh/id_rsa
docker exec master cat /root/.ssh/id_rsa.pub > temp_key.pub

docker exec worker1 mkdir -p /root/.ssh
docker cp temp_key.pub worker1:/root/.ssh/authorized_keys

docker exec worker2 mkdir -p /root/.ssh
docker cp temp_key.pub worker2:/root/.ssh/authorized_keys

rm temp_key.pub
```

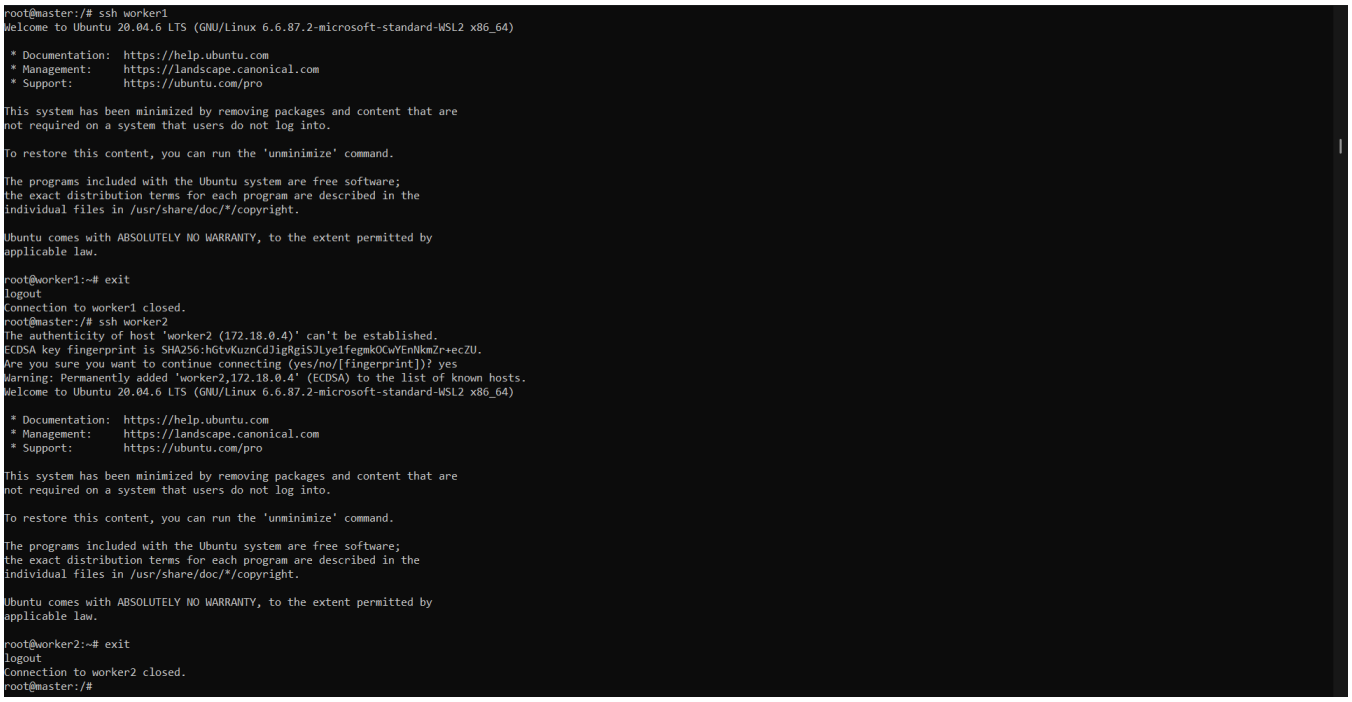
Bước 3: Kích hoạt quyền đăng nhập root qua SSH và khởi động lại SSH service trên cả 3 container (Ubuntu mặc định chặn root login qua SSH):

```
for node in master worker1 worker2; do
  docker exec $node bash -c "chown -R root:root /root/.ssh && \
    echo 'PermitRootLogin yes' >> /etc/ssh/sshd_config && \
    echo 'PubkeyAuthentication yes' >> /etc/ssh/sshd_config && \
```

```
    service ssh restart"
done
```

Bước 4: Kiểm tra đăng nhập chéo từ Master sang Worker mà không cần mật khẩu:

```
docker exec -it master bash
# Trong container master:
ssh worker1
ssh worker2
```



Hình 26: Kiểm tra đăng nhập SSH không mật khẩu từ Master đến Worker1 và Worker2 thành công

3.3 Giai đoạn 3: Cài đặt Hadoop trên Master

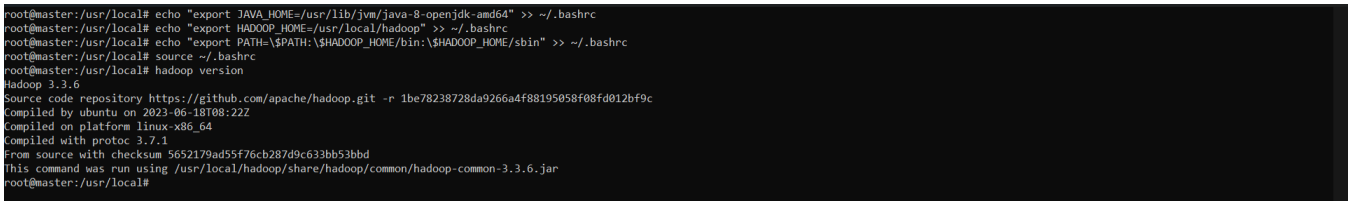
Bước 1: Tải Hadoop 3.3.6 trực tiếp từ máy chủ phân phối chính thức của Apache bằng lệnh wget:

```
docker exec master wget
↪  https://dlcdn.apache.org/hadoop/common/hadoop-3.3.6/hadoop-3.3.6.tar.gz -P /usr/local/
```

Bước 2: Giải nén và thiết lập biến môi trường:

```
# Trên container master:
cd /usr/local
tar -xzvf hadoop-3.3.6.tar.gz
mv hadoop-3.3.6 hadoop
rm hadoop-3.3.6.tar.gz

echo "export JAVA_HOME=/usr/lib/jvm/java-8-openjdk-amd64" >> ~/.bashrc
echo "export HADOOP_HOME=/usr/local/hadoop" >> ~/.bashrc
echo "export PATH=\$PATH:\$HADOOP_HOME/bin:\$HADOOP_HOME/sbin" >> ~/.bashrc
source ~/.bashrc
```



Hình 27: Lệnh `hadoop version` xác nhận đã cài Hadoop 3.3.6 thành công trên container Master

3.4 Giai đoạn 4: Cấu hình các file XML cốt lõi

Trên container **master**, tôi cấu hình lần lượt các file cốt lõi của Hadoop:

1. File workers - chỉ định danh sách Worker node:

```
cat <<EOF > /usr/local/hadoop/etc/hadoop/workers
worker1
worker2
EOF
```

2. File hadoop-env.sh - khai báo đường dẫn Java:

```
echo "export JAVA_HOME=/usr/lib/jvm/java-8-openjdk-amd64" >> \
  /usr/local/hadoop/etc/hadoop/hadoop-env.sh
```

3. File core-site.xml - chỉ định Master làm NameNode:

```xml
<configuration>
  <property>
    <name>fs.defaultFS</name>
    <value>hdfs://master:9000</value>
  </property>
</configuration>
```

4. File hdfs-site.xml - cấu hình số bản sao và thư mục lưu trữ:

```xml
<configuration>
  <property>
    <name>dfs.replication</name>
    <value>2</value>
  </property>
  <property>
    <name>dfs.namenode.name.dir</name>
    <value>file:///usr/local/hadoop/data/namenode</value>
  </property>
  <property>
    <name>dfs.datanode.data.dir</name>
    <value>file:///usr/local/hadoop/data/datanode</value>
  </property>
</configuration>
```

5. File yarn-site.xml và mapred-site.xml: Cấu hình Master làm ResourceManager và chỉ định framework MapReduce, tương tự như đã làm ở Phần 1.

3.5 Giai đoạn 5: Nhân bản cấu hình sang Worker

Thay vì cấu hình lại từ đầu trên mỗi Worker, tôi đẩy toàn bộ thư mục Hadoop từ Master sang 2 Worker bằng SCP (qua kết nối SSH đã cấu hình):

```bash
scp -r /usr/local/hadoop worker1:/usr/local/
scp -r /usr/local/hadoop worker2:/usr/local/
scp ~/.bashrc worker1:~/
scp ~/.bashrc worker2:~/
```

3.6 Giai đoạn 6: Xử lý sự cố phân quyền Root

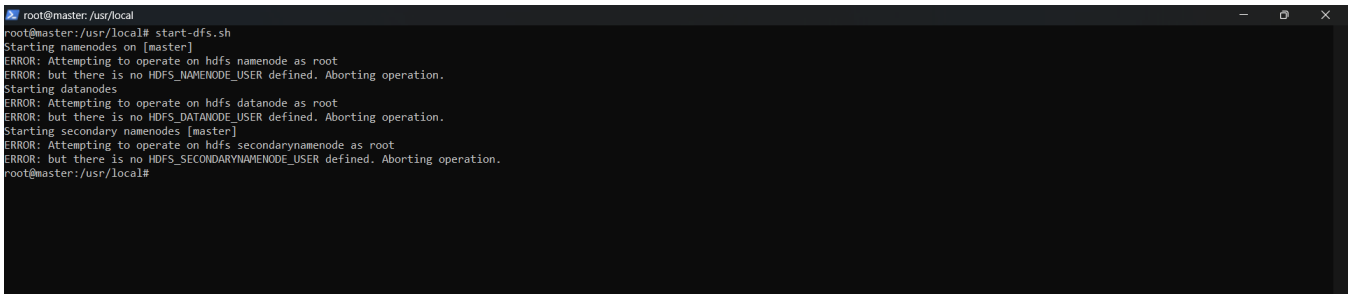
Format NameNode:

```
hdfs namenode -format
```

Sự cố: Lỗi phân quyền Root khi chạy HDFS

Khi gõ lệnh `start-dfs.sh` lần đầu, hệ thống văng lỗi:

```
ERROR: Attempting to operate on hdfs namenode as root
ERROR: but there is no HDFS_NAMENODE_USER defined. Aborting operation.
```



Hình 28: Lỗi phân quyền Root: Hadoop 3.x mặc định cấm chạy dịch vụ bằng quyền root

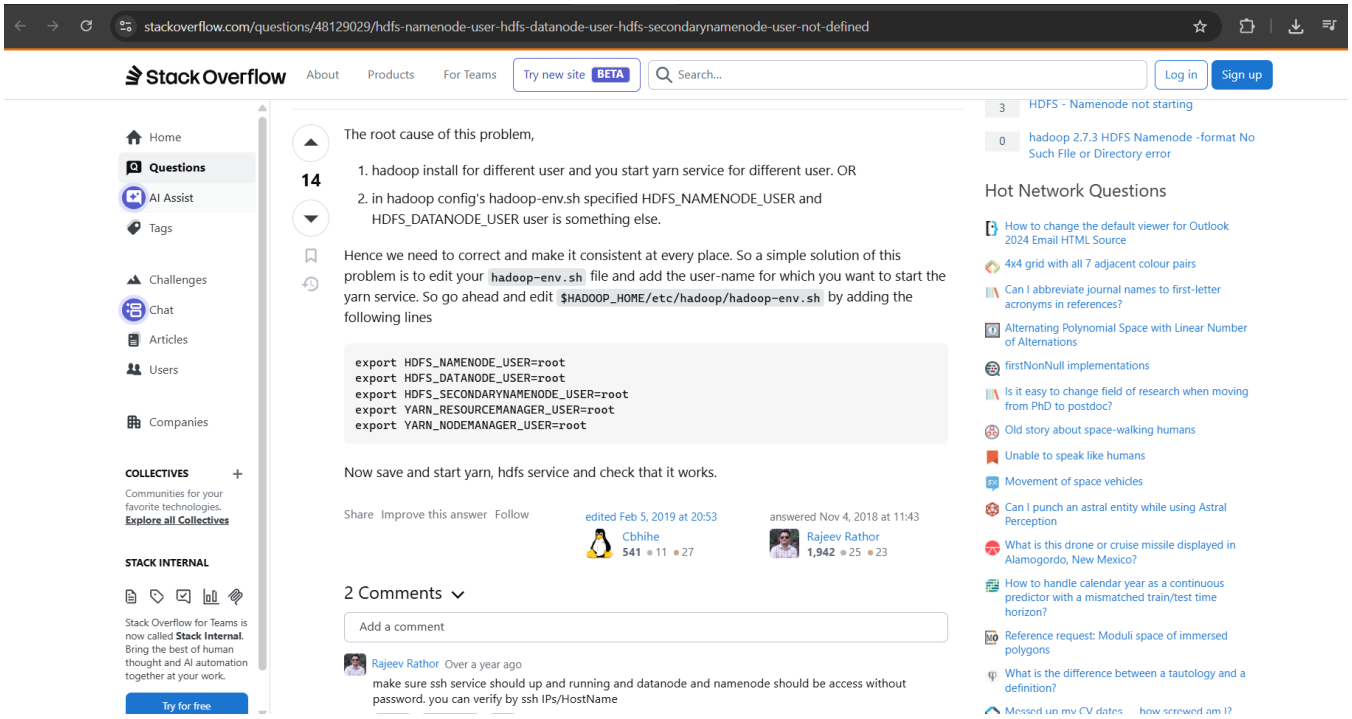
Nguyên nhân: Hadoop 3.x mặc định cấm chạy dịch vụ bằng quyền root vì lý do bảo mật. Docker chạy toàn bộ bằng root, nên cần khai báo tường minh.

Giải pháp: Bổ sung các biến môi trường vào `hadoop-env.sh` và đồng bộ sang các Worker:

```
cat <<EOF >> /usr/local/hadoop/etc/hadoop/hadoop-env.sh
export HDFS_NAMENODE_USER=root
export HDFS_DATANODE_USER=root
export HDFS_SECONDARYNAMENODE_USER=root
export YARN_RESOURCEMANAGER_USER=root
export YARN_NODEMANAGER_USER=root
EOF

# Đồng bộ file vừa sửa sang 2 worker
scp /usr/local/hadoop/etc/hadoop/hadoop-env.sh \
  worker1:/usr/local/hadoop/etc/hadoop/
```

```
scp /usr/local/hadoop/etc/hadoop/hadoop-env.sh \
  worker2:/usr/local/hadoop/etc/hadoop/
```



Hình 29: Quá trình tra cứu và giải quyết lỗi phân quyền Root trên diễn đàn StackOverflow

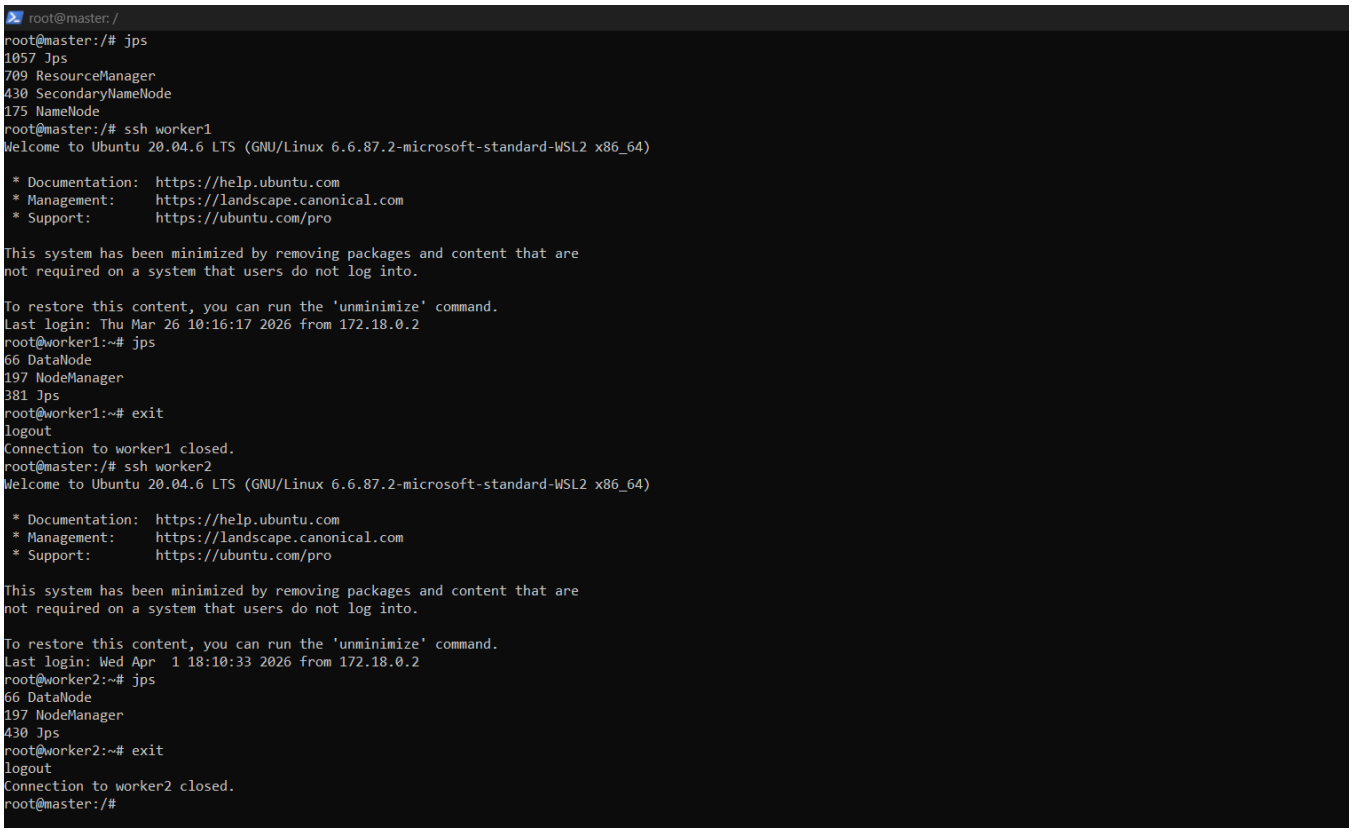
3.7 Giai đoạn 7: Khởi chạy cụm và kiểm tra

Sau khi xử lý xong sự cố phân quyền, tôi khởi động toàn bộ cụm từ container Master:

```
start-dfs.sh
start-yarn.sh
```

Kiểm tra sức khỏe cụm bằng `jps` trên từng node:

- **Trên Master:** lệnh `jps` hiển thị 4 tiến trình: `NameNode`, `ResourceManager`, `SecondaryNameNode`, `Jps`.
- **Trên Worker 1 & 2:** lệnh `jps` hiển thị 3 tiến trình: `DataNode`, `NodeManager`, `Jps`.



```
root@master:/
root@master:/# jps
1057 Jps
709 ResourceManager
430 SecondaryNameNode
175 NameNode
root@master:/# ssh worker1
Welcome to Ubuntu 20.04.6 LTS (GNU/Linux 6.6.87.2-microsoft-standard-WSL2 x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.
Last login: Thu Mar 26 10:16:17 2026 from 172.18.0.2
root@worker1:~# jps
66 DataNode
197 NodeManager
381 Jps
root@worker1:~# exit
logout
Connection to worker1 closed.
root@master:/# ssh worker2
Welcome to Ubuntu 20.04.6 LTS (GNU/Linux 6.6.87.2-microsoft-standard-WSL2 x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.
Last login: Wed Apr  1 18:10:33 2026 from 172.18.0.2
root@worker2:~# jps
66 DataNode
197 NodeManager
430 Jps
root@worker2:~# exit
logout
Connection to worker2 closed.
root@master:/#
```

Hình 30: Kết quả lệnh `jps` trên cả 3 node: Master, Worker1 và Worker2

4 Kết luận

Qua bài lab này, tôi đã trải qua toàn bộ chu trình cài đặt, cấu hình và vận hành một cụm Hadoop - từ chế độ pseudo-distributed trên môi trường WSL2 cho đến mô hình fully-distributed mô phỏng bằng Docker. Đây là nền tảng thực hành không thể thiếu để hiểu sâu hơn về kiến trúc của hệ sinh thái Big Data.

Những kiến thức thu được:

- **Kiến trúc HDFS:** tôi hiểu rõ vai trò phân công giữa NameNode (quản lý metadata và namespace) và DataNode (lưu trữ dữ liệu thực tế dạng block), cũng như cơ chế replication giúp đảm bảo tính chịu lỗi (fault tolerance). Nguyên tắc *write once, read many* của HDFS phù hợp với các workload phân tích dữ liệu lớn.

- **Hệ thống YARN:** tôi nắm được mô hình quản lý tài nguyên hai tầng của YARN - ResourceManager toàn cục điều phối tài nguyên, còn ApplicationMaster từng job tự quản lý lifecycle

của các task. Kiến trúc này tách biệt quản lý tài nguyên khỏi quản lý job, giúp YARN hỗ trợ nhiều framework trên cùng một cluster.

- **Mô hình Hadoop MapReduce thuần bằng Scala:** tôi xây dựng pipeline Map-Reduce với các lớp Mapper, Reducer và `CustomSortComparator` tùy chỉnh. Cách tiếp cận này giúp hiểu rõ hơn bản chất của thuật toán phân tán Shuffle & Sort trong Hadoop.
- **Ảo hóa với Docker:** việc sử dụng Docker Bridge Network để mô phỏng môi trường multi-node giúp tiết kiệm tài nguyên phần cứng mà vẫn tái hiện được đầy đủ kiến trúc phân tán của Hadoop fully-distributed.

Khó khăn và bài học kinh nghiệm:

Hai khó khăn chính tôi gặp phải là lỗi `NoClassDefFoundError` do Hadoop không tự nhúng thư viện Scala (giải quyết bằng Fat JAR với `sbt-assembly`) và lỗi `MRAppMaster` do biến môi trường YARN chưa trỏ đúng về thư mục Hadoop (giải quyết bằng bổ sung cấu hình `mapred-site.xml`). Ngoài ra, lỗi phân quyền root khi chạy Hadoop trong Docker cũng là bài học thực tế về sự khác biệt giữa môi trường sản xuất và môi trường phát triển container hóa.

Kinh nghiệm thực hành qua bài lab này là bước đệm quan trọng để tiếp cận các chủ đề nâng cao hơn trong môn học, bao gồm tối ưu hóa job Spark, tuning cấu hình YARN, và tích hợp Hadoop với các hệ thống lưu trữ và xử lý hiện đại như Hive, HBase, và Kafka.